Tekstprogrammering med Python

Programmeringsdidaktikk og matematisk modellering

Programmering med Python



Forord

Dette veiledningsheftet er et supplement for programmeringskurset for ENT3R-mentorer arrangert for ENT3R.

Med stor respekt vil jeg gjerne berømme arbeidet som legges ned av Nasjonalt senter for Realfagsrekruttering. Den gode jobben ENT3R-mentorer gjør for å inspirere ungdommene til å drive med realfag, og ikke minst for å skape et konkurransedyktig, bærekraftig og likestilt samfunn, er beundringsverdig.

Anbefaling til leseren

Når du skal lære et nytt emne er det ikke nok å pugge eller lære seg å få tak i hvordan ting skal gjøres. Det er like viktig å spørre seg hvorfor metoden er som den er, og forstå hvordan ting henger sammen. Da blir det lettere å lære. Jo bedre du forstår hvert delemne, desto lettere er det å bruke den til å løse aktuelle problemer. Du får da god erfaring til å anvende den i andre fagfelt.

Takk
Vi vil gjerne herved takke ENT3R-mentorene om med sitt viktige arbeid kan hjelpe elevene å komme et steg nærmere til sine drømmer. En stor takk rettes også til Jonas Gahr Sturtzel Lunde og Kristine Baluka ved UiO, og Jarle Peder Berntsen, Anita Gjesteland og Erik Andreas Hanson ved UiB, som har gitt oss tilgang til deres notater, modellerings og programmeringsoppgaver i oppgavesamlingene deres ved UiO og UiB. Jeg vil spesielt sende en stor takk til Marthina Sætre Vahedian, Lorentz T. Syslak, og alle ENT3R-mentorene i Bergen for korrekturlesing av teksten og innspill. Sist og ikke minst vil vi gjerne takke Øivind Wiborg og Alexander Westre Skog ved Bergen kommune, Erik Andreas Hanson og mine kollegaer ved HVL, for å lese gjennom dette heftet, og ha hjulpet å forbedre både teksten og innholdet.

Jeg vil gjerne takke Utdanningsdirektoratet som har gitt oss lov å bruke deres notater og figurer på https://www.udir.no/.

Bergen, 25. februar 2025
Amir M. Hashemi

Eventuelle kommentarer eller meldinger om feil i notatene tas imot med glede: ahas@hvl.no.

ISBN 9788294170029
© Forfatterne 2025–2030

Innhold

1 Hvordan programmering i skolen? 1

1.1 Viktige begrep i programmering . . . 2
1.2 Koding og programmering . . . 3
1.3 Programmering og programmeringsdidaktikk . . . 4

2 Litt programmering med Python 5

2.1 Å komme i gang med Python . . . 5
2.2 Noen viktige elementer i Python-progrmmering . . . 6
2.3 Noen av Pythons innebygde funksjoner . . . 7
2.4 Grunnleggende Python-programmering . . . 8
2.5 Datatyper . . . 12
2.6 Variabler . . . 13
2.7 Å sende inn datasett i Python-program . . . 14
2.8 Importering av pakker (numpy, math, pylab, sympy) . . . 16
2.9 Indeksering . . . 18
2.10 Løkker (betingelser og gjentagelser) . . . 19
2.11 Funksjoner . . . 25
2.12 Plotting i Python . . . 32
2.13 Lister . . . 34
2.14 Python-filhåndtering . . . 36
2.15 Laste inn, håndtere og plotte datasett . . . 37
2.16 Modellering: tekopp-temperatur . . . 37
2.17 Oppsummering . . . 40

3 Programmeringsdidaktikk 44

3.1 PRIMM-metoden for å tilrettelegge tekstprogrammering . . . 44
3.2 Hvordan man kan ta programmering i klasserommet og programmeringsdidaktikk 45
3.3 **DEL 1:** Å integrere algoritmisk tenkning med enkle eksempler . . . 46
3.4 Integrere og etablere algoritmisk tenkning i egne fag . . . 46
3.5 Å integrere programmering i egne fag . . . 47
3.6 Hvordan kan vi integrere algoritmisk tenkning/modellering/programmering i våre fag? . . . 49
3.7 **DEL 2:** Programmeringsoppgaver (skoletrinnet) . . . 52
3.8 Python- litt oppsummering . . . 54
3.9 Oppsummering - Enkle programmeringsoppgaver . . . 58
3.10 Programmeringsppgaver (Python) . . . 61
3.11 **DEL 3:** Programmeringsdidaktik (å ta programmeringen til klasserommet) . . . 71
3.12 Halveringsmetoden for å løse $f(x) = 0$ . . . 78
3.13 Elev-metoder for å løse ligningen $f(x) = 0$ . . . 81

Kapittel 1

Hvordan programmering i skolen?

I dagens høyteknologiske og rask skiftende verden blir det stadig tydeligere at elevene må kunne tenke kritisk og løse komplekse og ikke veldefinerte problemer som relateres til hverdagen. Mens få vil argumentere for nytten av å undervise i kritisk tenkning og problemløsningsferdigheter i skolene, er det mindre enighet om hvordan du skal gjøre det, når du skal begynne, eller hvilke begreper du skal bruke når du tilretteleggee i disse viktige kompetansene.

I forrige kapittel ble det løftet frem en rekke argumenter på hvorfor vi bør implementere programmering i den norske skolen. De viktigste prinsippene bygger på det verdenskjente rammeverket *21st century skills*, som forsøker å definere hvilke egenskaper og ferdigheter som vil være nødvendig for elever å beherske i fremtiden. Ved en raskt skiftende verden som stadig preges av en høyt akselererende teknologisk utvikling, vil det være mye som tilsier et økende behov for teknologisk kompetanse i det 21.årundre. Det predikteres også behov for kompetanser innenfor problemløsning, utforskning og kritisk tenkning for å kunne imøtekomme morgendagens muligheter og utfordringer. Til tross for positive tilbakemeldinger, støtte og stor enighet om hva som vil være fremtidens kompetansebehov, er det samtidig ikke gitt en entydig beskrivelse på hvordan elevene skal lære dette i skolen. Et lignende problem kan knyttes til resultater fra høringen av fagfornyelsen til læreplanverket for matematikk [**?**]. Funn tilsier at norske lærere har positiv holdning til å implementere programmering i skolen, men at de samtidig opplever usikkerhet knyttet til hvordan programmering skal tilrettelegges og inkluderes i undervisningen. I dette kapittelet vil vi derfor redegjøre for hvordan programmering kan integreres i undervisningen, og introdusere begreper som er relevant i forbindelse med dette.

1.1 Viktige begrep i programmering

Algoritmisk tenkning blir i økende grad anerkjent som en viktig pedagogisk ferdighet som er karakteristisk for læring i det 21. århundre. De senere årene har ordet begrepet «koding» blitt stadig mer utbredt i dagligtale, spesielt i samtaler som omhandler hvordan barn og unge som skal lære seg å programmere. Programmering kan knyttes til alt som dreier seg om å løse et problem ved hjelp av en algoritme eller et program. Dette innebærer blant annet forarbeid, planlegge, skrive, tegne, bearbeide, undersøke og teste problemløsningsmetoden, eksperimentere, revurdere, argumentere for at algoritmen alltid virker, generalisere algoritmen til å løse en hel klasse av problemer og undersøke effektivitet. Figuren forsøker å illustrere hvordan programmering avhenger algoritmisk tenkning og at programmering er mye mer enn bare koding.

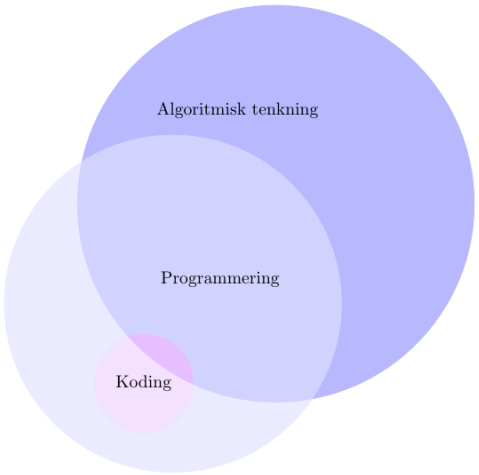


Fig. 2.1 Programmering og koding

Programmering kan knyttes til alt som dreier seg om å løse et problem ved hjelp av en algoritme eller et program. Programmering i skolen nevnes gjerne ofte i samme åndedrag som algoritmisk tenkning. Dette kommer trolig av hvordan programmering ofte beskrives om det å kommunisere ved hjelp av logikk og algoritmisk tenkning. Basert på denne definisjonen, fremkommer *koding* som selve prosessen og arbeidet med å reformulere dette inn i et bestemt *programmeringsspråk*.

Algoritmisk tenkning utgjør et viktig nøkkelprinsipp i naturvitenskapelig praksis og matematikk. I skolen har vi derfor lenge jobbet arbeidet med å tilrettelegge for at elever skal kunne utvikle utvikle elevenes ferdigheter innenfor til algoritmisk tenkning. Begrepene algoritmisk tenkning, programmering og koding blir i enkelte tilfeller og sammenhenger brukt mye om i noen sammenhenger om hverandre. Programmering er de konkrete skrittene som er nødvendig for å kunne skrive en kode. Programmering innebærer dermed å legge opp en plan (utforme pseudokode), bryte ned problemet i delproblemer og foreta selve kodingen. For å kunne redegjøre for hvordan programmering kan integreres i undervisning, må vi først avdekke nøkkelbegreper som er relevant i forbindelse med dette.

1.2 Koding og programmering

Koding og programmering utgjør er de to viktigste tilnærmingene innenfor programvareutviklingsvitenskapen. Vi vil derfor utdype de to begrepene nærmere i dette delkapittelet.

- **Programmering** beskrives om prosessen i å utvikle et kjørbart program på datamaskinnivå som kan implementeres uten å gi feilmeldinger. Det er prosessen med å skrive koder formelt slik at menneskelige innganger og tilsvarende maskinutganger forblir synkronisert.
- **Koding** omfatter er bare den delen av programmeringsprosessen som handler om å sende oppgaver og instrukser til datamaskin hva den skal gjøre. Koding i skolen dreier seg om at elevene lære elever til å løse problemer og samarbeide på kreative måter.

Visste du at vi bruker algoritmisk tenkning daglig? Dersom du tar en liten titt rundt livet ditt, så vil du fort innse at alt kan knyttes til et sett av spørsmål og svar. Hvordan kan du lage noe hvis du er sulten? Hvordan kan barna dine gå på skolen? Hvordan kan du forbedre deg selv til treningsstudioet? Hvordan kan du handle med et begrenset beløp? De fleste av våre daglige aktiviteter implementeres trinn for trinn og i henhold til hva som er førsteprioriteten. Dette gjøres ved å anvende logisk analyse, dekomponering, abstraksjon, mønstergjenkjenning og algoritmer, som alle utgjør hovedbegreper i algoritmisk tenkning.

Programmering utgjør mer enn å bare implementere allerede eksisterende algoritmer. Det handler om en prosess eller et samspill mellom problemløsning og metoder/algoritmer for å løse en hel klasse av problemer. Det å finne en generell metode utgjør den viktigste delen av prosessen, noe som gjør matematikkfaget til en naturlig arena. Dette gjelder uavhengig om digitale verktøy inngår i arbeidet eller ikke. Et viktig poeng som er verdt å se, og som gjerne ofte misforstås, handler om hva som selve målet med å lære programmering er. Fokuset bør ikke rettes mot å kunne beherske flest mulig programmeringsspråk, men heller forstå essensen bak de ulike delprosessene. Slik dannes det rom for utvikling og forbedring av ferdigheter innenfor logisk og algoritmisk tenkning, problemløsning og modellering. En slik forståelse utgjør en viktig forutsetning for å lykkes med å implementere programmering i lærings- og undervisningssammenhenger.

Kompetansemål i læreplan matematikk 1-10 trinn [25] og matematikk fellesfag Vg1 teoretisk [26] kan hjelp oss bedre å forstå hvilke kompetansemål forventes at elevene skal utvikle og dette kan hjelpe skolen og lærere å sette konkrete og klare læringsmål og planlegge hvordan de skal tilrettelegge programmeringen realfag og programfag. Med kjerneelementer mener vi både det viktigste innholdet, og det elevene må lære for å kunne mestre og bruke faget. De fastsatte kjerneelementene beskriver sentrale begreper, metoder, tenkemåter, kunnskapsområder og uttrykksformer, kan bidra å forstå hva som danner grunnlaget for utviklingen av læreplanene [10].

Algoritmisk tenkning kan betraktes som en generell kompetanse, og gjennom algoritmisk tenkemåte skal grunnlaget for programmering dannes. Ved arbeid med algoritmisk problemløsning kan lærere tilrettelegge at elevene få muligheter til å utvikle seg innenfor alle kjerneelementene, og læreren får mulighet til å vurdere elevenes kompetanse og få innsikt i hvordan de bruker sin kunnskap til å argumentere, reflektere, analysere og resonnere.

1.3 Programmering og programmeringsdidaktikk

Målet med dette heftet er å belyse hvordan vi best kan planlegge og tilrettelegge for matematikk og fysikk i skolen, og undersøke hvordan vi kan innlemme programmering i disse fagene.

I de neste kapitlene vil begrepene algoritmisk tenkning, problemløsning, modellering, programmering og sammenhengen mellom disse bli presentert. Med dette ønsker vi å bidra til å gi et faglig grunnlag for programmering, i tillegg til å støtte læreres kompetanse innenfor programmeringsdidaktikk.

Figuren nedenfor forsøker å illustrere hvordan man kan utforme og sette opp læringsmål ved hjelp av å ta utgangspunkt i kompetansemålene for hvert fag. Deretter kan det utformes en liste av delemner hvor programmering kan integreres. Dette kan bidra til å støtte og hjelpe lærerens arbeid med planlegging og tilrettelegging av undervisning hvor programmering inngår. Samtidig vil det trolig oppleves til å være mer eller mindre overkommelig å sette opp en slik liste, basert på hvilket fag man underviser i. Matematikk og fysikk er fag med kompetansemål som egner seg godt for innlæring gjennom programmering [22]. Programmering i skolen handler mye om logisk tenkning og problemløsning, noe som lett kan overføres til arbeidsoppgaver i flere fag. Algoritmetenkning er nyttig for matematikk og fysikkfagene, men det er også viktig å lære seg å tenke stegvis i form av sekvenser og konsekvenser i andre fag.

Programmeringsspråket, som stort sett brukes i ungdomsskolen og videregående, er *Python*, mens barneskolen bruker *Scratch*. Men det er viktig å lære tankegangen bak programmering, ikke programmeringsspråkene i seg selv. I de neste kapitlene diskuterer vi hvordan programmering kan brukes for å øke forståelsen i de ulike fagene.

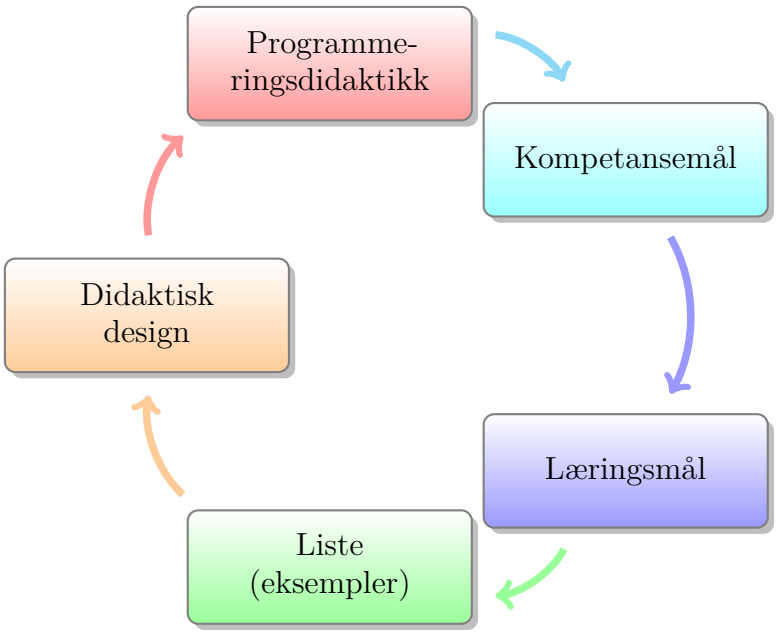


Fig. 2.2. Programmeringsdidaktikk med utgangspunkt i kompetansemål

Kapittel 2

Litt programmering med Python

Python er et objektorientert programmeringsspråk, som ble laget av Guido van Rossum på 80-tallet, og ble først presentert i 1991. Python er et generell programmeringsspråk som kan brukes på alle moderne operativsystemer. Python har en god del fordeler, blant annet er det enkelt å bruke og har mange innebygde funksjoner. Python har en enkel syntaks som ligner på det engelske språket. Python er et ganske moderne programmeringsspråk som kan brukes i forskjellige vitenskapelig kontekster (f.eks. i maskinlæring). Den kan brukes til behandling av tekst, tall, bilder, vitenskapelige data og omtrent alt annet du kan bruke på en datamaskin. Den brukes daglig i driften av f.eks. Googles søkemotor, videodeling YouTube, NASA og New York Børs.

2.1 Å komme i gang med Python

Det er to hovedversjoner av Python, Python 2 og Python 3. Disse versjonene er ganske forskjellige. Her bruker vi Python 3, fordi den er mer semantisk korrekt og støtter nyere funksjoner.
Python er et tekstbasert programmeringsspråk, og for å komme igang med å programmere, så har man flere alternativer blant annet:

- Programmere på nett: Det er flere tjenester på nettet som lar deg skrive og kjøre python-programmer, som for eksempel:

  https://trinket.io/python3

  https://replit.com/new/python3

- Installere en kildekode-plattform som både er en kode-editor, men som har innebygget en egen «Python-motor».

  https://www.spyder-ide.org/

  https://jupyter.org/

  Eksempel på et slikt program er Spyder. Dette er en åpen kildekode-plattform integrert i utviklingsmiljø for vitenskapelig programmering med Python. Spyder integreres med en rekke fremtredende pakker i den vitenskapelige Python-pakken, inkludert numpy, scipy, Matplotlib, pandas, IPython, sympy og Cython, samt annen programvare med åpen kildekode. Det finnes også andre slike program:

  https://www.anaconda.com/distribution/

  https://codewith.mu/en/download

Konsollvinduet og script i editorvinduet

I dette heftet har vi valgt å bruke Spyder. Vinduet til venstre er editorvinduet og nederst på høyre side er konsollvinduet. I Spyder kan man kjøre Python-kode direkte i konsollvinduet. Dette passer for små og enkle beregninger, for eksempel hvis vi bare skal regne ut et regnestykke som $2 + 5$,

```
In[1]: 2+5
Out[2]: 7
```

Men når man skriver lengre programmer, er det enklere å skrive koden i et *script*. Et Python-script er en kjørbar fil som inneholder Python-kode. Det vil si at hvis man skriver inn 2+5 i et script, så skjer det ikke noe før man kjører scriptet. Her er et eksempel på et script som skal beregne arealet til et rektangel:

```
a = 5
b = 3
# Regner ut de to rottene
A= a*b
print('Arealet til et rektangel med sidelengdene', 'a =', a, 'og b=
    ', b, ' er lik', A)
```

Dersom vi kjører dette scriptet, vil vi få:

```
Arealet til et rektangel med sidelengdene a = 5 og b= 3 er lik 15
```

online-programmering

Det er anbefalt av flere læresteder å bruke:

https://trinket.io/python3

når man skal programmere på nettet.

2.2 Noen viktige elementer i Python-progrmmering

De viktige elementene i Python-scriptet er:

- *Kommentarer:* Når man programmerer, så lønner det seg å forklare hva koden gjør. Det gjør man med tegnet *#kommentar* eller 'kommentar' eller kommentar". Dette tegnet tolker Python som «resten av denne linjen skal ikke leses av programmet». Kommentarer kan også settes mellom enten 'enkle apostrofer' eller doble apostrofer".
- *Streng:* En streng er en sekvens av tegn som er ment å tolkes av Python som tekst.
- *Funksjoner:* en gruppe kodelinjer, som utfører en spesifikk oppgave, for eksempel beregner en matematisk uttrykk eller skriver ut en tekst. En funksjon kan brukes flere sted i ett program, med å kalle funksjonen.
- *Pakker:* For å utføre beregninger, er det nødvendig å importere en passende pakke. Her importerer vi pakken `numpy` som `np`, slik at vi kan bruke for eksempel funksjonen `sqrt` som tilhører pakken som `np.sqrt`.

- *Lister:* Når vi trenger å lagre flere verdier i samme variabel, kan vi benytte lister. Lister er samlinger med tall, strenger, variabler eller annet.
- *Datatyper:* En datatype forteller kompilatoren (tolkeprogrammet) hvilken datatype, som behandles (tekst (`str`), tall (`int, float`), vektor (`array`)).
- *Variabler:* Videre definerer vi de tre koeffisientene a, b og c. I Python kalles disse *variabler*, og likhetstegnet angir verdien deres. Også x_1 og x_2 som vi så regner ut, er variabler, og vi ser at i utregningen brukes a, b og c, som har verdiene vi allerede har gitt dem.
- *Indeksering:* Indeksering er prosessen som ordner et sett med data slik at det blir mulig å spore opp datene. Indeksering i Python starter ved indeksen 0.
- *Løkker:* Løkker brukes for å gjenta noe mange ganger etter hverandre

2.3 Noen av Pythons innebygde funksjoner

Python-interpreter har en rekke innebygde funksjoner som alltid kan kalles med å skrive deres navn med riktige input som inngår i funksjonene.

Funksjon	Beskrivelse
hjelp( )	Kaller det innebygde hjelpesystemet
print()	Skriver ut data
abs()	Returnerer absoluttverdi av et tall
float()	Returnerer et flyttall
int()	Returnerer et heltall
input()	For å få input fra bruker
input(float())	For å få input (et reelt tall) fra bruker
input(int())	For å få input (et heltall) fra bruker
input(str())	For å få input (tekst) fra bruker
len()	Returnerer lengde av et objekt
list()	Returnerer en liste
range()	Returnerer en sekvens av hele tall, som standard starter fra 0 og inkrementerer tallene med 1
round()	Returnerer rundverdi av et tall

2.4 Grunnleggende Python-programmering

Kommentere i koden med ' ' og

Kommentarer (en forklarende tekst) er det første vi vanligvis møter i koden. En kommentar starter enten med en hash, og varer ut linjen, eller skrives inne i triple anførselstegn, som første linje i eksempelkoden over. Ved triple anførselstegn kan kommentaren vare over flere linjer. Dette er tekst eller kode som ignoreres når koden kjøres. Det er lurt å ha med kommentarer underveis i koden, fordi det gjør den enklere å lese, og det kan være til hjelp når du senere ser tilbake på den.

```
# -*- coding: utf-8 -*-
"""
Created on Fri Dec 16 12:39:51 2022
@author: ahas
"""
print('Skriv ut noe')
```

Ved å kjøre programmet får vi:

```
Skriv ut noe
```

Strenger

En streng er en sekvens av tegn som er ment å tolkes av Python som tekst.
Man kan skrive tekst (strenger) både med apostrof ('... ') og med anførselstegn ("..."):

```
print('Denne linjen blir skrevet ut')
print(''Denne linjen blir skrevet ut'')
```

Man får ikke feilmelding ved å bruke ekstra mellomrom:

```
print('Denne linjen blir skrevet ut')
print ('Denne linjen blir skrevet ut')
```

Man får feilmelding hvis man glemmer noe av syntaksen man skal ta med:

```
print(Denne linjen blir skrevet ut')
```

Vi skal se nærmere i notatene at print-funksjonens tolker følgende som streng og som tall:

```
print('3+4')
print(3+4)
```

Resultatet av den første programsetningen blir 3+4 og den andre 7.

Oppg. 2.4.1: a) Skriv en programkode som skriver ut: Denne setningen skrives ut!

b) Finn feilen i følgende programsetninger:

```
Print('Jeg liker programmering')
print('Jeg liker programmering!)
```

Oppg. 2.4.2: Hva blir resultat av følgende programsetninger:

a)

```
print('5+7')
```

b)

```
print(5+7)
```

c)

```
print('5+7)
```

d)

```
Print('5+7')
```

e)

```
print('5+7')
print()
```

Kalkulator

Vi ønsker at kalkulatoren skal kunne addere (+), subtrahere (-), multiplisere (*) og dividere (/). Vi kaller +, -, * og / for operatorer.

Matematiske operatorer

Operator	Utfører	Eksempel	Resultat
$+$	Addisjon	$5+2$	7
$-$	Subtraksjon	$5-2$	3
$*$	Multiplikasjon	$5*2$	10
$/$	Divisjon	$5/2$	2.5
$//$	Heltalldivisjon	$13//3$	4
**	Eksponent	2**3	8
%	Modulo	$13 \div 4$	1

• a/b gir kvotienten når divisoren a deles på dividenden b.
• $a//b$ gir antall heltall som er i kvotienten (når man deler a på b) når divisoren a deles på dividenden b.
• $a\%b$ gir restleddet når divisoren a deles på dividenden b.

Heltallsdivisjon er divisjon uten restledd. Dette betyr at for eksempel både $14//7$ og $15/7$ blir 2, siden $14 : 7 = 2$ og $15 : 7 = 2 + \frac{1}{7}$.
• $a \div b$ (modulo) gir restleddet når man deler a på b.

Eksempel 2.4.1: Her er et eksempel fra konsollvinduet:

```
> 2+3+4*5
25
>3/2
1.5
>2**3 +1
9
> 15/7
2.142857142857143
>15//7
2
```

Oppg. 2.4.3: Regn ut disse med Python

a) $3 \cdot 5 + \frac{4}{5}$

b) $2^3 + 3^2 + \frac{1.25}{5}$

c) Hva resultatet til 18%4 blir?

Oppg. 2.4.4: Løs disse oppgavene v.h.a. Python

a) Regn ut arealet til en trekant med høyden 5 cm og grunnlinjen 8 cm.

b) Regn ut arealet til et parallellogram som har høyden 7 cm og grunnlinjen 10 cm.
Arealet til et parallellogram er altså lik lengden (grunnlinjen) multiplisert med høyden.

c) Regn ut arealet til en trapes som har høyden 6 cm og lengden til de to parallelle sidene er 8 cm og 12 cm.
Arealet av et trapes regnes ut ved å multiplisere høyden og den gjennomsnittlige lengden av de to parallelle sidene.

Å hente π fra numby-biblioteket

Eksempel 2.4.2: Vi ønsker å bestemme arealet til en sirkel med radius 5 cm.
Her har vi to muligheter Vi kan regne direkte med *pi* som verdien 3.14, og regne direkte på kommandolinjen:

```
> 3.14*5**2
78.5
```

eller vi kan definere pi som 3.14 og regne ut arealet et skript:

```
pi=3.14
r=5
Areal = pi*r**2
print(Areal)
```

Ved å kjøre programkoden får man 78.5.
Vi lærer senere at vi kan importere pi fra *math*- eller *numpy*-biblioteket, der man enten installer hele *math*- eller *numpy*-biblioteket eller bare installerer det man trenger. Du kan bruke *numpy*-bibliotekets *numpy.pi* eller *math.*bibliotekets for *math.pi* og få verdien av den matematiske konstanten π i python.

```
import numpy
r=5
Areal = numpy.pi*r**2
print(Areal)
```

```
import numpy as np

r=5
Areal = np.pi*r**2
print(Areal)
```

Hvis man skal regne ut med kjente verdier som π eller kjente funksjoner som cos, kan man enten installere hele *numpy*-biblioteket:

```
from numpy import *
r=5
Areal = pi*r**2
print(Areal)
```

eller bare innstallere det man trenger:

```
from numpy import pi

r=5
Areal = pi*r**2
print(Areal)
```

Hvis man importerer fra numpy bare det man trenger, kan bare bruke dem uten å bruke np. Tilsvarende gjelder dette også for funksjoner som vises i det neste eksempelet.

2.5 Datatyper

En datatype forteller kompilatoren (tolkeprogrammet) hvilken datatype som behandles, og hvilke regler som skal brukes for hvordan dataene skal behandles. Her har vi fokus bare på enkle datatyper, datatyper som bare inneholder 'en verdi. De mest brukte er:

- int - integer (heltall som 1,2,3)
- float - desimaltall (eks 2.5 - OBS: her må man bruke punktum og ikke komma)
- str (string) - tekst, OBS: skrives mellom anførselstegn
- bool - boolean (True/False - OBS: når man skal gi en variabel verdien True eller False, må de skrives med stor forbokstav)

I tillegg har vi noen sammensatte datatyper, hvor hver datatype kan inneholde flere verdier, f.eks.

- list - lister
- tuple - tupler/tuppel (ligner på lister, men er en ikke-muterbar sekvens - den kan ikke endres og brukes først og fremst på data som ikke skal endres, slik som ukedager og måneder)
- set - mengder
- dict - dictionary (oppslagsliste)

Syntaks	Utfører	Eksempel
str	streng (tekst)	'matematikk'
int	heltall	37
float	desimaltall	3.14
list	liste	['k', 1, 2, 'A']
set	mengde	$\{1, 3, 5\}$
bool	boolsk verdi	True, False
np.array	np-array–datastruktur	array([1, 2])

Eksempel 2.5.1: Her vises et eksempel, hvordan strenger oppfører seg med matematiske operasjoner:

```
print('tre' + 'ti')
print('to' * 3)
```

Utskriften blir da:

```
treti
tototo
```

2.6 Variabler

Som i matematikk brukes variabler i programmering. Variabler brukes til å huske data til bruk senere i programmet, for eksempel til utregninger, beslutninger og for å gjøre endringer på informasjon. Variabler kan innta verdier eller tekst, og for tekst skal ha med apostrof ('... ') eller anførselstegn ("...") rundt seg. Man kan opprette en variabel i et program ved å skrive en bokstav eller en tekst ettefulgt av et liketstegn og

– verdien som variabelen skal innta hvis variabelen skal være *et tall*
– apostrof ('... ') eller anførselstegn ("...") rundt seg (åpnes og avsluttes) hvis variabel skal være *tekst/streng*.

Som vist nedenfor her:

```
x= 2
y= 'Tekst'
z= "Tekst"
```

I løkkene blir ofte variablene oppdatert ved å legge til et tall til variabelen eller multiplisere variabelen med et tall, for eksempel:

Eksempel 2.6.1: Her settes variabelen n til verdien 2 og blir deretter oppdatert:

```
n = 2
# det legges til 4
n = n+4
# Resultatet ganges med 5
n=5*n
# Resultatet legges til 4
n += 4
# Resultatet ganges med 5
n* = 5

print('n blir da',n)
```

Her blir verdien til variabelen n satt sammen med 4 og ganget med 5 på to ulike måter, før resultatet blir multiplisert med 5 og blir vist. Svaret er da $((2+4)*5+4)*5 = 170$. ◊

Oppg. 2.6.1: Skriv en programkode som utfører følgende regneoperasjoner ved hjelp av variabler:

a) Skriv en programkode som leser inn to tall m og n som tar verdiene 7 og 1 henholdsvis, deretter blir m summert med 3 og n ganges med 5, og til slutt multipliseres m og n sammen og resultatet blir skrevet ut på skjermen som: *Produktet er* og resultatet.

b) Skriv en programkode som leser inn to tall a og b som tar tallet 5 og teksten *TALL*, deretter multipliseres a og b med henholdsvis 2 og 3 og de oppdaterte variabelene a og b blir vist på skjermen

Oppg. 2.6.2: Skriv en programkode som beregner volum og overflateareal til en rektangulær rett prisme med lengde 5 metere, bredde 4 meter og høyde 3 meter.

Oppg. 2.6.3: Skrive en programkode som regner ut $y = h - \frac{1}{2}gt^2 + v_0 t$ for $h = 3.5$, $g = 9.81$, $v_0 = 20$ og $t = 5$, og skriver på skjermen.

Avrunding

F-literalene bruker vi når vi vil runde av et tall i en *print-kommando*. Rett før høyre mengdeparentes legger vi inn et kolon. Da kan vi styre hvordan utskriften formateres. Vi runder av til to desimaler med formatdefinisjonen «.3f». Her betyr bokstaven «f» at variabelen skal skrives ut med et fiksert antall desimaler, og tretallet betyr at det skal være tre desimaler. Hvis du i stedet skriver «.2f», får du to desimaler.

Eksempel 2.6.2: Her er et eksempel, hvordan to tall og deres sum blir avrundet:

```
a = 25/7
b = 32
print(f"Her er a {a:.3f} og b {b:.1f}. Summen er {a+b:.3f}.")
```

Det er vanlig at resultatet fra programmet som skrives til skjermen, inneholder både tekst og verdier av variabler. I programmet bruker vi det som kalles f-literal. Vi setter en «f» rett etter venstreparentesen for å fortelle at det kommer en f-literal. Alt som står inni mengdeparenteser i en f-literal, blir tolket som Python-kode. Det vil si at a blir tolket som verdien av variabelen a.

2.7 Å sende inn datasett i Python-program

1. I dataprogram kan variablene deklareres med å definere variablene som får inndelt en verdi:

```
r=2
n=5
```

2. Et dataprogram kan ta imot data fra brukeren hver gang man prøver å kjøre programmet:

```
alder=input("Skriv inn alderen din:")
print(f''Du er {alder} år.'')
```

   Her kunne man angi alderen som tall eller tekst. Hvis man ønsker seg å presisere hvilken datatype (heltall, flyttall og tekst), vil man ta i bruk videre i programmet følgende:

```
alder=int(input("Skriv inn alderen din:"))
r = float(input("Skriv inn radiusen til sirkelen (et reelt tall)
    her: "))
n = int(input("Skriv inn a (et heltall) her: "))
```

3. Når vi skal behandle datafil i Python, er det første vi må gjøre å laste ned en fil med dataene og lagre den i et hensiktsmessig format. De vanlige formatene er .txt og .csv. Når vi laster ned filer med stort tallmateriale, er det ofte lurt å bruke .csv (se avsnitt 0.14).

I neste eksempel vises hvordan `float(input())` og `float(input())` for å lese inn takk og hvordan `print(f("tekst og {tall}")` kan brukes for å formatere det som skrives ut på skjermen.

Formatering av tekst og tall

Her tas et eksempel som viser hvordan man kan lese inn hele og reelle tall fra brukeren, og skriver ut på skjermen, og hvordan man kan avrunde og formatere i utskriften:

```
print('Her leses inn variablene')
a = int(input("Skriv inn a (et heltall) her: "))
b = float(input("Skriv inn b (et reelt tall) her: "))

c= a+b
print()
print(f'Summen til {a} og {b} er: {c} ')
print() # lager en linje mellomrom
print(f'Summen til {a} og {b} er:c=', c)
print()
print('formatering:')
print(f'Her er a {a:.2f} og b {b:.1f}. Summen er {c:.3f}.')
print(f'Summen til {a} og {b} er: {c:.3e} ')
```

Ved å kjøre programmet får vi da på skjermen

```
# Her leses inn variablene

Skriv inn a (et heltall) her: 11
Skriv inn b (et reelt tall) her: 2.35

Summen til 11 og 2.35 er: 13.35
Summen til 11 og 2.35 er: c =13.35

formatering:
Her er a 11.00 og b 2.4. Summen er 13.350.
Summen til 11 og 2.35 er: 1.335e+01
```

Hvis man skulle lese tekst fra brukeren, kunne man bruke `str(input(()).`

Oppg. 2.7.1: Skrive en programkode som oppretter tre variabler a, b og c der a blir gitt som input av brukeren, og $b = 3$ og $c = 5$. Programmet skal regne ut $d = \frac{1}{3}ab + c^2$, og skrive resultatet på skjermen.

Oppg. 2.7.2: Skrive en programkode som oppretter to variabler r og h som blir gitt som input av brukeren (radius og høyden til en rett sylinder), og som regner ut $V = \pi r^2 h$ (volumet til sylinderen, $\pi = 3.14$), og skriver resultatet på skjermen.

2.8 Importering av pakker (numpy, math, pylab, sympy)

En pakke eller et bibliotek i Python er en samlig av moduler, hvor en modul er en fil som inneholder Python-definisjoner (funksjonsdefinisjoner), men inneholder ikke kall til. Modul kan være en en samling av funksjoner og klasser som kan gjenbrukes i mange ulike programmer. Når man bruker en modul fra en pakke, skrives: `pakke.modul`, henter modulen `modul` fra pakken `pakke`. Det finnes mange ulike pakker man kan benytte i Python, og vi skal hovedsakelig bruke pakkene `numpy` (numerisk Python) og `matplotlib` her.
Disse to kommandoene for å importere hele pakken er ekvivalente:

```
import numpy
```

```
from  numpy import *
```

Eller man kan bare importere det man trenger:

```
from  numpy import pi
```

Noen foretrekker å bruke:

```
import numpy as np
```

der for eksempel for å bruke π skriver man i programmet *np.pi*.
Under følger et kort eksempel der vi importerer noen pakker, og bruker dem til en liten beregning av $y(x)$ for en gitt x-verdi.

Eksempel 2.8.1: Vi ønsker å beregne $y(x) = e^{-\pi x}$ for $x = 0.2$.

```
import numpy as np
x= 0.2
y = np.exp(-np.pi*x)

print('y er lik',y)
```

Eksempel 2.8.2: Man kan importere hele pakken eller deler av pakken:

```
import numpy as np      # Importerer hele pakken som får navnet np
# Vi vil bruke pakkens cosinus- og pi-funksjoner:

print("sinus av ",np.pi," er ", np.sin(np.pi))
print()  # oppretter ekstra linje

# Vi kan ogsaa  bare importere enkeltfunksjoner. Da kan vi kalle paa
   funksjonene uten aa angi pakkens navn.

from numpy import sin, pi    # importerer vi bare delene vi trenger.

print("sinus av ",pi," er ", sin(pi))   # Merk at vi ikke trenger
   pakkens adresse.
```

Oppg. 2.8.1: Skrive en programkode som regner ut $y = \sin(\pi x)$ for $x = 0.5$.

Eksempel 2.8.3: Vi vil regne ut $\sin(1) + \cos(4) + e^{-2}$.

```
import numpy as np
svar=np.sin(1)+np.cos(4)+np.exp(-2)
print(svar)
```

```
from numpy import exp, sin, cos
svar=sin(1)+cos(4)+exp(4)
print(svar)
```

nympy eller *math*-biblioteket?

numpy-biblioteket inneholder flerdimensjonale array- og matrise-datastrukturer, mens *math*-biblioteket brukes til å utføre matematikk på skalardata, for eksempel trigonometriske beregninger. Hvis man skal regne ut med kjente funksjoner som cos, kan man enten importere hele *numpy*-biblioteket eller bare importere det man trenger:

```
from numpy import exp, sin, cos, tan, arcsin, arccos,arctan, exp
svar=sin(1)+cos(4)+exp(4)
print(svar)
```

Husk at de trigonometriske funksjonene (sin, cos og tan) tar inn en vinkel i radianer, og inversfunksjonene (arcsin, arccos og arctan) returnerer en vinkel i radianer.

Oppg. 2.8.2: Regn ut disse v.h.a Python

a) $\cos(2) + \tan(1.5)$

b) $e^4 + \log(2) + \ln(0.1)$

pylab-biblioteket

I de nye matematikk-lærebøkene for videregående-trinnet brukes *pylab*-biblioteket, som har mange innbygde funksjoner. *pylab*-pakken inneholder både *numpy* og *matplotlib*.

```
from pylab import *
print(cos(1))
```

Noen foretrekker numpy og at man bruker

```
import numpy as np
```

der man for eksempel for å bruke sin-funksjonen skal skrive $np.sin(x)$. Fordi man har da et bedre oversikt hvor i programmet har brukt pakken.

Oppg. 2.8.3: Skrive en programkode som regner ut $y = \sin(\pi x)$ for $x = 0.5$.

nympy- eller *sympy*-biblioteket?

numpy-biblioteket gjør beregningsmatematikk, mens *sympy*-biblioteket gjør symbolsk matematikk. For eksempel for å derivere eller å integrere:

```
from sympy import *

x = Symbol('x')
svar1= diff(x**3, x)
svar2= integrate(svar1, x)
print(svar2)
```

Andre pakker (*Matplotlib*)

Vi vil ofte ha behov for å visualisere data. Da kan biblioteket *Matplotlib* hjelpe oss. Her skal vi bruke modulen *pyplot* (som er en del av biblioteket Matplotlib) for å tegne en graf:

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  x = np.linspace(-3,3, num = 101)
5  f = x**3-3*x
6
7  plt.plot(x,f, '--')
8  plt.grid()
9  plt.xlabel('x')
10 plt.ylabel('y')
11 plt.legend(['f(x)'])
12 plt.savefig('MinPythonFigMappe/eksplot.png')
```

2.9 Indeksering

Vi lager indeksmengder med kommandoen range. Kommandoen range(2, 5) betyr heltallene fra og med 2 til, men ikke med, 5 (til og med 4).

range(1,n+1)	alle hele tall fra og med 1 til n
range(1,n+1,m)	alle hele tall fra og med 1 til n med m steglengde
range(1,n+1,m)	alle hele tall fra og med 1 til n med m steglengde

Hvis du utelater første argument i range, antar Python at første argument er 0. Dersom du legger til et tredje argument, angir dette steglengden i intervallet.

Kommando	Verdi
range(0, 5)	0, 1, 2, 3, 4
range(5)	0, 1, 2, 3, 4
range(2, 5)	2, 3, 4
range(0, 5, 2)	0, 2, 4
range(10, 5, -1)	10, 9, 8, 7, 6

2.10 Løkker (betingelser og gjentagelser)

En løkke brukes for beregninger eller handlinger som skal gjentas flere ganger. Man kan velge et gitt antall ganger eller at det skal gjentas inntil et mål (noe) er nådd. Fordelen med å bruke en løkke er at vi slipper å skrive mange setninger med programkode. Da blir koden enklere å feilsøke og rette, men den blir også mer kompakt, noe som gjør at det tar kortere tid å kjøre gjennom programmet.

if/elif/else-løkke

En if/elif/else-setning brukes når vi ønsker å utføre en blokk med kode når den gitte tilstanden er sant eller usant. Her er et eksempel:

```
a = float(input("Skriv inn a (et reelt tall) her: "))
if a> 2:
    print("a er mer enn 2")

elif a<2:
    print("a er mindre enn 2")

else:
    print("a er lik 2")
```

Ved å kjøre programmet og taste inn forskjellige verdier for a får vi:

```
Skriv inn a (et reelt tall) her: 4
a er mer enn 2
Skriv inn a (et reelt tall) her: 1
a er mindre enn 2
Skriv inn a (et reelt tall) her: 2
a er lik 2
```

Oppg. 2.10.1: Hvis -7 skrives inn når følgende programkoden kjøres, hva blir resultatet?

```
print('Skriv et tall')
a = int(input())
x=a
if x < 0:
    x = -x
print('Absluttverdien til',a,' er', x)
```

Oppg. 2.10.2: Skriv en programkode for følgende:

a) Les et tall fra skjermen, og sjekk om tallet er positivt, negativt eller null, og skriv resultatet på skjermen.

b) Les et tall fra skjermen, og sjekk om tallet er partall eller oddetall, og skriv resultatet på skjermen.

c) Les to tall fra skjermen og si om disse tallene har samme eller motsatt fortegn.

while-løkke

While-løkker brukes i tilfeller hvor man kan si at løkka skal utføres så lenge noe er sant.

```
i=0
while i < 3:
    print("i =", i)
    i += 1
```

Ved å kjøre programmet får man:

```
i = 0
i = 1
i = 2
```

Andre ganger vil vi gjøre bestemte utregninger så lenge en betingelse vi har satt på forhånd, er oppfylt. Vi skal vise bruken av denne type løkke ved hjelp av et eksempel. Legg merke til at $range(1, n+1)$ er alle hele tall fra og med 1 til n.

Eksempel 2.10.1: Skriv et program som regner ut a_2, a_3, $\cdots$ a_{10} når: $a_1 = 1$ og $a_{n+1} = 0.25a_n + 2$. Hva konvergerer a_n når $n \to \infty$?

Løsning:

```
n = 1
for i in range(1,10):
    0.25*n + 2
    print(n)
```

Når man kjører programmet for de første 10 leddene, ser man at $a_n \simeq 2.666...$ når $n \to \infty$. ◊

Eksempel 2.10.2: Skriv et program som regner ut a_2, a_3, $\cdots$ a_{10} når: $a_1 = 1$ og $a_{n+1} = \dfrac{3}{4 - a_n}$. Hva konvergerer a_n når $n \to \infty$?

Løsning:

```
n = 1
for i in range(1,10):
    n = 3/(4-n)
    print(n)
```

Når man kjører programmet for de første 10 leddene, ser man at $a_n \simeq 1.0$ når $n \to \infty$. ◊

Eksempel 2.10.3: Betrakt summen $\displaystyle\sum_{n=1}^{\infty} \frac{5}{n}$.
Finn ut hvor mange ledd som trengs for å få summen over 20.

Løsning:

```
n = 1  # Definerer n og Sum ved start
Sum = 0
  # Vil regne ut summeringen til summen er mindre eller lik 20
while Sum <= 15:
   Sum += 5/n
   n += 1
   print('Summen passerer 15 etter', n, 'ledd. Da er summen', Sum)
```

Ved å kjøre programmet får man:

```
Summen passerer 15 etter 32 ledd. Da er summen 20.136225977182594
```

Oppg. 2.10.3: Skriv kort hva følgende programkode utfører:

```
innlest = input("Hva er 49+35?")
tall = int(innlest)

if tall != 84:
    while tall != 84:
        innlest = input("Regn igjen!")
        tall = int(innlest)
    else:
        print("Du klarte det!")
```

Oppg. 2.10.4: Skriv en programkode som spør hva summen til $52 + 19$ er, og leser inn svaret fra brukeren, og hvis svaret er feil, skriver ut *Feil. Regn igjen!*, og hvis det er rett, skriver ut *Du klarte det!*

Oppg. 2.10.5: Følgende programkode spør leseren om å skrive et partall, og deretter sjekker om dette tallet er partall, og bestemmer summen til alle partall til og med dette tallet.

```
tall = int(input("Skriv inn et partall: "))

if tall%2 == 1:
    print("Tallet er ikke et partall!")
else:
    sum = 0
    i=0
    while(i <= tall):
        sum = sum + i
        i = i+2

print(sum)
```

Skriv en programkode som gjør dette for oddetall.

Oppg. 2.10.6: Vi vil prøve å finne det minste heltallet x (heltall) for at $3^x > 81$.
Følgende program vil telle hvor mange ganger, må gange 3 med seg selv for å få mer enn 81.

```
x0= 1
b = 81
x = 0
x1 = x0

while x1 <= b:
    x += 1
    x1*= 3

print(f"Svaret er {x}.")
```

Skriv programmet og sjekk om du får svaret 4.
Modifiser programmet for å gjøre dette for ligningen $4^x = 1024$

for-løkke

for-løkken kan brukes til å gjenta en kodeblokk et kjent/begrenset antall ganger, eller gå gjennom en sekvens, element for element:

```
n=2
for i in range(1,n+1):
        b=i**2+1
        print(b)
```

Ved å kjøre programmet får man:

```
2
5
```

Legg merke til at *range(1,n+1)* betyr indeksmengde fra og med 1 til og n (og ikke til og med $n+1$).

Oppg. 2.10.7: Skriv hva resultatet av følgende programkode blir:

a)
```
i=0
for i in range(2):
    print('A')
    for i in range(3):
        print('B')
```

b)
```
for x in [50, 60]:
 for y in [1, 2, 3]:
     print(x+y, end=' ')
```

c)
```
for x in [50, 60]:
 print()
 for y in [1, 2, 3]:
     print(x+y, end=' ')
```

Oppg. 2.10.8: Hva får man av følgende programkoder? Skriv følgende i et script og kjør programmet.

a)
```
for i in range(0,10):
    print(i, end=" ")
```

b)
```
for i in range(1,10):
    print(i, end=" ")
```

c)
```
for i in range(0,10,2):
    print(i, end=" ")
```

d)
```
for i in range(10,1,-1):
    print(i, end=" ")
```

Oppg. 2.10.9: Skriv en programkode for følgende: Det dukker på skjermen: *Hei, vil du fortsette med programmet?* Hvis svaret er nei, avsluttes programmet.

Oppg. 2.10.10: Skriv en programkode som regner ut summen til de første 5 leddene i en geometisk rekke med førsteledd 3 og kvotient 2, og skriver ut på skjermen summen.

Oppg. 2.10.11: Skriv kort hva følgende program utfører:

```
for i in range(1,10):
    for k in range (1,5):
        print(i,k)
```

Oppg. 2.10.12: Skriv en programkode som regner ut summen til de første 7 leddene i en aritmetisk rekke med førsteledd 5 og kvotient 3, og skriver ut summen på skjermen.

Oppg. 2.10.13: a) Skriv hva resultatet til følgende programkode blir:

```
for i in range(1,2):
    for k in range (1,3):
        print(i,2*k)
```

b) Skriv også hva `range(5)` sender inn i løkken.
Skriv hva resultatet til programkoden blir

```
for i in range(5):
    if i == 3:
        print(i, end="")
    else:
        print(2*i, end="")
```

c) Skriv også hva `while i < 4` sender inn i løkken.

```
i = 1
while i < 4:
  print(i)
  if i == 3:
    break
  i += 1
```

Hvordan ville du endre løkken i del c) hvis du ville bruke `range`-syntaksen?

d)

```
 = 1
while i < 5:
  print(i)
  i += 1
else:
  print("i er ikke lenger mindre enn 5")
```

e) Hva skriver følgende programkode ut?

```
r = 4
for i in range(1, r + 1, 1):
    for j in range(1, i + 1):
        print(j, end=' ')
    print()
```

f) Hva skriver følgende programkode ut?

```
n=4;
for i in range(n):
    for j in range(i):
        print ('o ', end="")
    print('')

for i in range(n,0,-1):
    for j in range(i):
        print('* ', end="")
    print('')
```

g) Skriv en programkode som får følgende resultat:

```
*
* *
* * *
o o o o
o o o
o o
o
```

Oppg. 2.10.14: Følgende programkode

```
n = 4
k = 4
for i in range(0,n+1):
    for j in range(k-i,0,-1):
        print(j,end=' ')
    print()
```

Skriver ut:

```
4 3 2 1
3 2 1
2 1
1
```

a) Gjør de nødvendige endringene i programmet for at svaret blir:

```
1
1 2
1 2 3
1 2 3 4
```

b) Gjør de nødvendige endringene i programmet for at svaret blir:

```
4 3 2 1
3 2 1
2 1
1
```

c) Gjør de nødvendige endringene i programmet for at svaret blir:

```
1
2 1
3 2 1
4 3 2 1
```

d) Gjør de nødvendige endringene i programmet for at svaret blir:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

2.11 Funksjoner

Funksjoner brukes til å dele opp programmer i mindre deler. I programmering er funksjoner en måte å samle en beregning utført av en eller flere variabler i en sammensatt bolk, såkalte *funksjoner*, slik at det blir lettere å skrive, modifisere, lese koden, bruke andre steder i programmet eller eventuelt i andre program. Syntaksen for funksjoner er kontrollordet *def*, deretter navnet på funksjonen etterfulgt av to parenteser som innholder variabel (variabler), som inngår funksjonen og et kolon (:).

```
def funksjonsnavn(variabler):
  kodeblokk
```

Husk å trykke tab (knappen rett over caps lock) for linjene i kodeblokken for å få innrykk. Funksjoner gjør programmeringen lettere.

Litt om funksjoner: Hva og hvorfor?

Funksjoner i Python (**Hva?**):

- ◊ består av en gruppe kodelinjer som utfører en spesifikk oppgave
- ◊ kan minne om matematiske funksjoner, men også Python-funksjoner
  - kan inneholde alle slags handlinger, ikke bare matematikk
  - kan returnere et svar (som mat.funk.), men kan også la være
  - kan ta argumenter, men kan også la være

Hvorfor bruke funksjoner?

- ☐ Dele et større program opp i mindre deler
  - lettere å lese, skrive og vedlikeholde kode
  - Lettere å forstå, enklere å teste, mer effektivt samarbeid
  - gir ryddigere og enklere kode
- ☐ Gjenbruk av kode
  - Vi kan bruke Pythons egne funksjoner, f.eks. print(), round()
  - Vi kan selv definere nye funksjoner og
    - bruke dem flere steder i ett program,
    - eller i flere ulike programmer

Her tar vi et eksempel på hvordan vi kan lage en funksjon som kan brukes flere ganger i programkoden. For eksempel kan vi skrive en funksjon som kan regne ut volumet av en sylinder med radius r og høyde h.

```
1 import numpy as np
2
3 def Volumsylinder(r, h):
4
5     volum=np.pi*(r**2)*h
6     return volum
7
8 V=Volumsylinder(3,5)
9 print(V)
```

I scriptet regner vi ut `Volumsylinder(3,5)` volumet av sylinderen med radius = 3 og høyde = 5. Funksjoner begynner med kontrollordet `def`, etterfulgt av navnet på funksjonen, parenteser og et kolon, der input-parameterne er gitt i parantes. Dersom du vil at funksjonen skal kunne ta ett eller flere argumenter, skriver du dem inn i parentesene. I dette eksemplet har vi deklarert variablene r (radius) og h (høyde), og det betyr at disse variablene inngår i funksjonen og de må defineres som et tall når funksjon skal regne ut volumet.

Legg merke til at etter funksjonshodet `def Volumsylinder(r, h):` kommer resten av koden i linjene under med innrykk (bruke TAB-tasten). Så utføres utregningen av volumet, før vi til slutt har med `return volum` som skal returnere svaret funksjonen gir, og vi kan bruke videre i koden. Istedenfor `Volumsylinder(3,5)`, kunne vi bruke:

```
a=3
b=5
V=Volumsylinder(a,b)
print(V)
```

Ellers variablene kan leses inn i programmet fra skjermen:

```
r= float(input('Skriv inn radiusen til en sylinder: '))
h= float(input('Skriv inn høyden til en sylinder: '))
V=Volumsylinder(r,h)
```

Eksempel 2.11.1: Her er 3 eksempler med funksjoner uten argument, med argumenter, og med matematiske beregninger og hvordan disse kan brukes (*kalles*).

```
def minfunk():
    print("Her er en funksjon!")

def minfunkmedarg(Navn, Hilsen):
    print("Hei %s,  %s" %(Navn,Hilsen))

def minfunktall(x,y):
    return x**2 + y**2

minfunk()

minfunkmedarg("Kari Olsen","Ha en fin dag!")

a = minfunktall(1,2)
print(a)
```

Ved å kjøre programkoden får vi:

```
Her er en funksjon!
Hei Kari Olsen, Ha en fin dag!
5
```

Eksempel 2.11.2: For eksempel setter følgende program opp funksjonen $f(x) = x^3 + 3x$ og lister opp på skjermen x og $f(x)$ under x og $f(x)$:

```
def f(x):
    return x**3 + 3*x

print(f'x\t f(x)')

for x in range(10):
    print(f'{x}\t {f(x)}')
```

Ved å kjøre programmet, ser resultatet slik ut:

```
x   f(x)
0 0
1   4
2   14
3   36
4   76
5   140
6   234
7   364
8   536
9   756
```

Eksempel 2.11.3: Her tas et eksempel med en funksjon som beregner skalarproduktet til to vektorer:

```
def skalarprod(a, b):
    p= 0
    for i in range(len(a)):
        p += a[i]*b[i]
    return p
# Prøv funksjonen:
a = [1, 2, 3]
b = [0, 3, -1]
ab = skalarprod(a,b)
print(f'Skalarproduktet av a og b er {ab:.2f}')
```

Ved å kjøre programmet, får man:

```
Skalarproduktet av a og b er  3.00
```

Oppg. 2.11.1: Hva får man ved å kjøre følgende program:

```
def skalarprod(m, n):
    d= 0
    for i in range(len(m)):
        d += m[i]*n[i]
    return d
# Prøv funksjonen:
m = [-1, 3, 0]
n=  [2, 1, 5]
mn = skalarprod(m,n)
print(f'Skalarproduktet av m og n er {mn:.1f}.')
```

Oppg. 2.11.2: a) Skriv et program som setter opp en funksjon for arealet til en sirkel med radius r og regn ut arealet til en sirkel med radius 5.

b) Skriv et program som leser inn radiusen til en sirkel fra brukeren og regner og skriver ut arealet til en sirkel.

Oppg. 2.11.3: Summene nedenfor beregner $\ln x$ når n velges stor nok.

$$s_1(x,n) = \sum_{i=1}^{n} (-1)^{i-1} \frac{1}{i} (x-1)^i, \quad 0 < x \leq 2$$

$$s_2(x,n) = \sum_{i=1}^{n} \frac{1}{i} \left(1 - \frac{1}{x}\right)^i, \quad x > \frac{1}{2}$$

Vi ønsker å lage en programkode som hjelper til å beregne disse summene for en gitt x og n. Vi vil studere estimeringsavviket (feilen) man få ved å bruke summen $s(x,n)$ for en gitt x, istedenfor $\ln x$. Estimeringsfeilen for s_1 kan studeres ved to forslag:

1) Å bestemme feilen ved å regne ut differansen mellom den eksakte verdien og den estimerte verdien: $\ln(1.5) - s(1.5,n)$ (naiv metode).

2) Å Regne ut det først leddet i summen s_1 som ikke er tatt med i beregningen av summen, som kan kalles det numeriske estimeringsavvikket. To programkoder er skrevet slik:
For s_2 kan man se nærmere på 1).

```
# Programkode 1
import numpy as np

def S1(x, n):
    S1 = 0
    for i in range(1,n+1):
         S1+= (-1)**(i-1)*(x-1)**i/i
    return S1

def E1(x, i):
    E1 = 0
    E1= (x-1)**i/i
    return E1

x = 1.5
n=100
e=0.000001

for i in range(1,n+1):
    EstimertLog1 = S1(x,i)
    Estimeringsfeil1 = E1(x,i)
    EksaktLog = np.log(x)
    NaivEstimeringsfeil = abs(EstimertLog1 -EksaktLog )
    if NaivEstimeringsfeil< e:
        break

print(f' Ved å bruke {i} første ledd: Estimert= {EstimertLog1},
   Eksaktverdi= {EksaktLog}, Naiv estimeringsfeil = {
   NaivEstimeringsfeil},Estimeringsfeil = {Estimeringsfeil1}' )
```

```python
# programkode 2
import numpy as np

def S2(x, n):
    S2 = 0
    for i in range(1,n+1):
        S2+= (1-1/x)**i/i
    return S2

x = 1.5
n=100
e=0.000000001

for i in range(1,n+1):
    EstimertLog2 = S2(x,i)
    EksaktLog = np.log(x)
    NaivEstimeringsfeil = abs(EstimertLog2 -EksaktLog )
    if NaivEstimeringsfeil< e:
        break
print(f'Ved å bruke {i} første ledd: Estimert= {EstimertLog2},
   Eksaktverdi= {EksaktLog }, Naiv estimeringsfeil = {
   NaivEstimeringsfeil}' )
```

Ved å kjøre programmeringskoder får man disse to visningene på skjermen (som ikke nødvendigvis er skrevet i samme rekkefølge som programkodene):

```
Ved å bruke 15 første ledd: Estimert= 0.4054657568451514, Eksaktverdi=
   0.4054651081081644, Naiv estimeringsfeil = 6.487369870189497e-07,
   Estimeringsfeil = 2.0345052083333333e-06
```

```
Ved å bruke 16 første ledd: Estimert= 0.40546510744296105, Eksaktverdi=
    0.4054651081081644, Naiv estimeringsfeil = 6.652033368581556e-10
```

a) Forklar hvilken programkode som kan brukes til å beregne $\ln(0.25)$, $\ln(1.75)$ og $\ln(2.25)$?

b) Hvordan kan man forbedre presisjonen og få bedre nøyaktighet?

Oppg. 2.11.4: Arealet til en trekant med sidelengdene a, b og c er gitt ved:

$$A = \sqrt{s(s-a)(s-b)(s-c)}$$

der $s = \frac{1}{2}(a+b+c)$.
Skriv en programkode som beregner arealet til en trekant for en gitt a, b og c.

Oppg. 2.11.5: Kinetisk energi for en partikkel med masse m [kg] som har hastighet v [m/s] er gitt ved:

$$E = \frac{1}{2}mv^2$$

Skriv en programkode som definerer en funksjon for kinetisk energi, regner ut den kinetiske energien for en partikkel som har massen $m = 0.0001$ [kg] og hastigheten $v = 8$ [m/s].

Å kalle inn en funksjon i en annen funksjon

Eksempel 2.11.4: Her tas et eksempel med en funksjon som finner det største tallet blant tre tall ved å kalle en funksjon i en annen funksjon

```
def maksavto( x, y ):
    if x > y:
        return x
    return y
def maksavtre( x, y, z ):
    return maksavto( x, maksavto( y, z ) )
print(maksavtre(2, -3, 7))
```

Eksempel 2.11.5: Her tas et eksempel med en funksjon som kalles inn i en annen funksjon. Skriv en programkode for en funksjon $f1(x)$ som multipliserer argumentet x med 4 og en annen funksjon $f2(x)$ som legger til tallet 5 til argumentet x. Utvid programkoden til å regne ut $f1(f2(2))$ og $f2(f1(2))$. Ellers formuleres dette slik også:
Gitt $f(x) = 4x$ og $g(x) = x + 5$. Skriv disse som funksjoner i programkode, og bruk programmet til å regne ut $f(g(2))$ og $g(f(2))$.

```
def f1(x):
    y= 4*x
    return y

def f2(x):
    y=x+5
    return y

print(f1(f2(2)))
print(f2(f1(2)))
```

Svaret ved å kjøre programmet blir:

```
28
13
```

Sjekker: $f1(f2(2))$, der $f2(2) = 2 + 5 = 7$ og $f1(f2(2)) = f1(7) = 4 \cdot 7 = 28$.
Sjekker $f2(f1(2))$, der $f2(f1(2)) = f1(2) = 4 \cdot 2 = 8$ og $f2(8) = 8 + 5 = 13$. ◊

Oppg. 2.11.6: Hva skrives ut som resultat av kjøring av følgende programkode?

```
def printut(a):
    b = 3*a
    print('b = %g' % b)
x= 5
print('x = %g' % x)
printut(x)
```

2.12 Plotting i Python

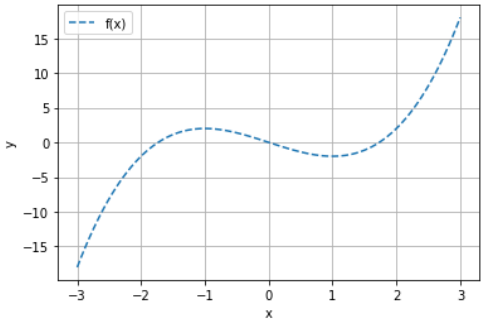
Vi vil ofte ha behov for å visualisere data. Da biblioteket *Matplotlib* hjelpe oss. Her skal vi bruke modulen pyplot (som er en del av biblioteket Matplotlib) for å tegne en graf:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-3,3, num = 101)
plt.yticks([-15,-5,5,15])
f = x**3-3*x

plt.plot(x,f, 'b--')
plt.xlabel('x')
plt.ylabel('y')
plt.legend(['f(x)'], loc='best')
plt.grid()
plt.show
plt.savefig('MinPythonFigMappe/eksplot.png')
```

Kjører vi dette scriptet, vil følgende figur plottes *og lagres*.



Elementene vi har tatt i bruk, her er følgende:

- `plt.plot(x,f,'b--')` tegner grafen til funksjonen og tar inn x-verdiene definert i `x = np.linspace()`, de tilhørende funksjonsverdiene, og deretter `'b--'`, som skifter utseendet på grafen til en stiplet i stedet for en sammenhengende graf med blåfarge. Flere argumenter kan angis i `plt.plot()` for å spesifisere detaljene til grafen.
- Funksjonen `np.linspace(start, end, num = N+1)` deler intervallet $[start, end]$ inn i N like deler med $N + 1$ jevnt fordelte punkter.
- `plt.yticks([-15,-5,5,15])` spesifiserer tallverdiene på y-aksen.
- `plt.grid()` legger på et gitter bak grafen.
- `plt.xlabel()` og `plt.ylabel()` definerer navn på aksene.
- `plt.legend()` legger til en tekstboks for å beskrive hva grafen heter. `loc='best')`: plasserer tekstboksen der det passer best.
- `plt.show` tegner diagrammet i konsollvinduet.
- `plt.savefig()` Denne funksjonen gjør at vi kan lagre figuren direkte når vi kjører koden. Her må vi legge til riktig path inne i en streng, slik at bildet blir plassert i den mappen vi ønsker.

KONTROLL SPØRSMÅL. Hvordan kan du endre graftykkelsen og farger på grafen? Hvordan kan du endre programkoden ved å skrive f ved hjelp av funksjonssyntaksen? ■

Oppg. 2.12.1: Følgende programkode er gitt:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-2,2, num = 101)
plt.yticks([-10,-5,5,10])
f = x**5-5*x**2

plt.plot(x,f, 'g--')
plt.xlabel('x')
plt.ylabel('y')
plt.legend(['f(x)'], loc='best')
plt.grid()
plt.show
plt.savefig('MinPythonFigMappe/EksGraf.png')
```

a) Forklar hva programmet utfører linje for linje.

b) Gjør de nødvendige endringene for at programkoden tegner grafen til: $g(x) = x^3 - 5x$ når $-2 \leq x \leq 2$.

Oppg. 2.12.2: La f være funksjonen gitt ved $f(x) = x^3 + 6x - 3$.

a) Skriv en programkode som setter opp verditabell for x-verdiene mellom -2 og 2 (med å dele opp intervallet med steglengde $= 0.01$), og tegner grafen til f.
Utvid programmet til å tegne grafen til funksjonen

b) Utvid programmet ditt til å la brukeren velge minste og største verdi for x og trinnene, i tabellen.

Oppg. 2.12.3: Skriv en programkode som tegner grafen til følgende funksjoner i samme koordinatsystem:

$$f(x) = sin(2x - \pi)$$

$$g(x) = cos(2x + \pi)$$

i intervallet $[-2\pi, 2\pi]$.
Skisser også $h(x) = cos(2x + \frac{\pi}{2})$. Hva kan vi si om f og h?
Utvid programmet for at den kan tegne grafen til funksjonene:

$$f(x) = c + a\sin\left(\frac{2\pi}{T}kx + d\right), \quad 0 \leq x \leq T$$

$$g(x) = c + a\cos\left(\frac{2\pi}{T}kx + d\right), \quad 0 \leq x \leq T$$

gitt $c = 60$, $a = 120$, $T = 365$, $k = 2$ og $d = 182.5$.

2.13 Lister

Lister er mengden av elementer atskilt med komma mellom hakeparenteser, for eksempel [1, 2, 3] og ['Klara', 'Augusta', 7]. Lister kan brukes til å lagre objekter underveis i programmet. For å lage en liste, oppretter man en tom liste ved å tilordne verdien [] til en variabel, for eksempel liste=[] eller xverdier=[]. Her er variabelnavnene «liste» og «xverdier».

liste = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']:

Kommando	Resultat	Forklaring
liste[0]	'a'	Første element er nr. 0
liste[0:3]	['a', 'b', 'c']	Tar ikke med nr. 3 (fjerde)
liste[:3]	['a', 'b', 'c']	Tar med element på plass nr. 0, 1 og 2
liste[-1]	'h'	Siste element er nr. -1
liste[3:6]	['d', 'e', 'f']	Fra og med nr. 3 til nr. 6
liste[3:-1]	['d', 'e', 'f', 'g']	Fra nr. 3, tar ikke med siste element
liste[3:]	['d', 'e', 'f', 'g', 'h']	Fra nr. 3, tar ikke med siste element

For å legge til elementer i en liste, brukes kommandoen *«append»*, og listevariabelen brukes som forstavelse, atskilt med et punktum. I koden nedenfor opprettes det først en tom liste i linje 1, og i de tre neste linjene legger vi inn tallene 3 og 1 og bokstaven «c». I linje 5 skriver vi lista til skjerm.

```
1 liste = []
2 liste.append(3)# legger tallet 3 inn i lista
3 liste.append(1)# legger tallet 1 inn i lista
4 liste.append('c')# legger bokstaven c inn i lista
5 print(liste)
```

Når vi kjører programmet, får vi dette resultatet:

```
1 [3, 1, 'c']
```

Kontrollordet *len* kan gi oss lengden på lista, *len(liste)*, altså hvor mange elementer den inneholder:

```
1 liste=[1,7,2,4,3,3,5,6]# en liste med sju elementer
2 print(len(liste))
```

Svaret bli da 8.

Eksempel 2.13.1: Skriv en programkode som gir [0, 1, 'e', 'p'].

```
1 liste = []
2 liste.append(0)
3 liste.append(1)
4 liste.append('e')
5 liste.append('p')
6 print(liste)
```

Man kan også legge til flere elementer samtidig med programkoden

```
1 liste = []
2 liste.extend([0,1,'e','p'])
```

Oppg. 2.13.1: Hva blir resultatet av følgende?

a) Programkoden:

```
A=[1,2,3,4,5]
B=[2,4,6,8,10]
C=A+B
print(C)
```

b) Programkoden:

```
A=[1,2,3]
A+=[4]
A = A +[5]
print(A)
```

c) En liste er gitt som: Liste=[1, 2, 3, 4, 5, 6] Bruk en for-løkke til å regne ut produktet av alle elementene i liste.

Oppg. 2.13.2: La f være funksjonen gitt ved $f(x) = -0.2x^2 + 1$. Skriv en programkode som setter opp x-verdiene, mellom -1 og 2 med steglengde 0.25, og y-verdiene til f inn i to lister.

Oppg. 2.13.3: La f være funksjonen gitt ved $f(x) = -x^3 + x + 2$. Skriv en programkode som setter opp x-verdiene, mellom -2 og 3 med steglengde 0.25, og y-verdiene til f inn i to lister.

2.14 Python-filhåndtering

I Python er det ikke nødvendig å importere eksternt bibliotek for å lese og skrive filer. Python har en innebygd funksjon for å lage, skrive og lese filer.

Filkommandoer	Forklaring
f = open('filnavn')	Åpner en fil, returnerer filreferanse
f = open('filnavn','tilgang')	Åpner en fil, med spesifisert tilgang
	F.eks. 'w' åpner en fil for skriving (se neste side)
f.read()	Returnerer hele innholdet av filen
f.read(n)	Returnerer n karakterer av innholdet
f.readline()	Returnerer neste linje (før / n)
f.readlines()	Returnerer hele innholdet av fila som en liste
f.write(s)	Skriver strengen s til fil
f.writelines(liste)	Skriver innholdet av liste av strenger til fil
f.seek(offset fra hvor)	Forflytter filpekeren (index) i filen
f.tell()	Returnerer posisjon til filpekeren i filen
f.close()	Lukker filen

Tilgangstyper for open

Filkommandoer	Forklaring	Tidl. data	Hvis filen ikke finnes
'r'	Lesing	Bevares	FEIL
'w'	Skriving	Fjernes	Oppretter ny fil
'a'	Skriving (bakerst)	Bevares	Oppretter ny fil
'r+'	Lesing og skriving	Bevares	FEIL
'w+'	Lesing og skriving	Fjernes	Oppretter ny fil
'a+'	Lesing og skriving (helt bakerst)	Bevares	Oppretter ny fil

```
f = open('datafil.txt','r')  # aapner fil for lesing
f = open('datafil.txt','w')  # aapner fil for skriving
f = open(r'C:\User\files\fil.txt','a' )
# r  først inni parentesen fo at \  ikke tolkes som spesielle tegn
# deretter leser filen fil.txt fra stien.
```

Eksempel 2.14.1:

```
f = open('fil1.txt', 'a')
f.write('Denne skrives i filen!')
print()
f.close()
print()
#åpen og les fra filen
f = open('fil1.txt', 'r')
print(f.read())
```

2.15 Laste inn, håndtere og plotte datasett

Vi har lært så langt hvordan vi bruke matematiske funksjoner til å regne funksjonsverdier og plotter disse. I naturvitenskapen vil ofte data / tall / målte verdier være hentet fra eksperimenter og observasjoner. Det kan ofte være store datasett med lange lister med tall. Vi skal nå hvordan vi kan laste inn data fra harddisken til vår pc eller eksterne servere og bruke innbygde Python-funksjoner og pakker for å håndtere slike lister.

pandas-biblioteket

pandas-bibliotet brukes til å analysere data.

```
import pandas as pd
df = pd.read_csv('data.csv')
```

I det neste eksempelet skal vi lese tid−temperatur-målinger, der vi bruker følgende syntakser for å åpne datafilen *tekopp-temp.csv* der i regnearket hver tekstlinje er en rad og verdiene for de ulike søylene er separert med komma (;). For å indikere dette, bruker man *sep=';'* og for å tolke desimaltall bruker man *decimal='.'*.

```
import pandas as pd
df = pd.read_csv('http://home.hvl.no/ansatte/ahas/MP/tekopp-temp.csv',
    sep=';', decimal='.', comment='#',encoding='latin-1)

tid = df['Tid'].tolist()
temperatur = df['Temperatur'].tolist()
```

2.16 Modellering: tekopp-temperatur

Prosjekt: Modellering av tekopp-temperatur

Vi ønsker å studere endringen i tekopp temperatur i løpet av en time (60 min).

Læringsmål

- Utforske sammenhengen mellom konstant prosentvis endring, vekstfaktor og eksponential-funksjoner
- Bruke funksjoner i modellering og argumentere for framgangsmåter og resultater,
- Modellere situasjoner knyttet til reelle datasett, presentere resultatene og argumentere for at modellene er gyldige
- Utforske matematiske egenskaper og sammenhenger ved å bruke programmering
- Tolke og bruke funksjoner i matematisk modellering og problemløsing.

Læringsutbytte

- Studentene skal forstå hvordan kurven som beskriver at temperaturfallet avtar raskest i starten for så å nærme seg temperaturen i rommet som er en asymptote til kurven.
- Studentene skal se at lineær kurvetilpasning ikke kan dekke behovet, spesielt i de første minuttene.

Programmeringsdidaktikk

Teori (det tas litt teori på tavla før eksperimentet begynner):

Newtons avkjølingslov: dette forsøket prøver å lage en matematisk modell som beskriver hvordan temperaturen endres i et objekt (en kopp med vann) som kjøles ned av omgivelsene. Funksjonsuttrykket:

$$y = f(t) = at + b$$

representerer en lineær funksjon som kan beskrive lineær sammenheng mellom to størrelser. Funksjonsuttrykket:

$$y = f(x) = c + ae^b t\,, \quad \text{der } b < 0$$

representerer en eksponentialfunksjon. Ved start (t=0) har funksjonen verdien $y = c + a$, og synker eksponentiell mot $y = c$ når tiden (t) går.

Utstyr:

- 10 eller flere kopper (samme type)
- Vannkoker
- En vekt
- En klokke
- Ett eller flere termometer(e)

Tidsbruk:
Forarbeid: 15-20 min og elevene jobber ca. 60 min, 10-15 min refleksjon over detaljene i prosjektet med åpen samtale og eventuelt kontrollspørsmål.

Gjennomføring:
Kok opp tilstrekkelig vann til å fylle opp alle koppene. Sett koppene på vekta etter tur og fyll opp med like mye av det varme vannet. Mål temperaturen med jevne mellomrom i løpet av en time (3600 sek), for eksempel etter 5 min (300 sek), 10 min (600 sek), ... og noter resultatet i tabellen.

Tid (sek)	Temp (Celsius)

For eksempel (data hentet fra time-temp.pdf (southampton.ac.uk))

Tid (sek)	Temp (Celsius)
0	81.50
310	62.50
620	45.70
910	36.80
1220	29.40
1810	25.30
2550	24.20
3100	23.30
3620	22.50

Eksempel 2.16.1: I dette eksempelet skal vi nå se på filen 'tekopp-temperatur.csv'. Hver linje i filen er en temperatur-måling i en tekopp i løpet av en time (3600 sekunder) som er plassert i et rom med temperatur 22 grader celsius. Her bruker vi Newtons avkjølingslov for å studere temperaturendringen i vannkoppen.

$$\frac{dT}{dt} = k(T - T_o)$$

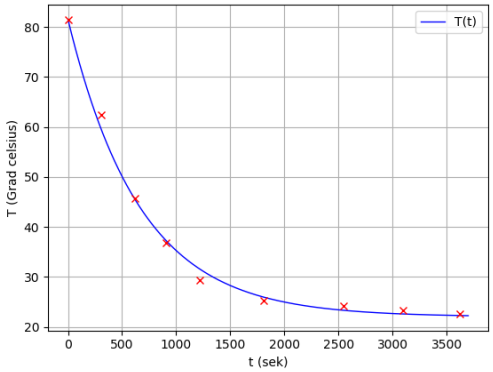
der $T(t)$ ((målt i grader celsius) er vanntemperatur i vannkoppen ved tiden t (målt i sekunder).

```
import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv('http://home.hvl.no/ansatte/ahas/MP/tekopp-temp.csv',
    sep=';', decimal='.', comment='#',encoding='latin-1)

tid = df['Tid'].tolist()
temperatur = df['Temperatur'].tolist()
tstart = 0
Tstart = temperatur[0]
tslutt =3700
deltat = 0.5
To = 22 # Rom temperatur
k = .0015 # proporsjonalitets koeff.
tverdier = []
Tverdier = []
t = tstart
T= Tstart
while t <= tslutt:
    tverdier.append(t)
    Tverdier.append(T)
    t = t + deltat
    deltaT = -deltat * k * (T - To)
    T = T + deltaT

plt.plot(tverdier, Tverdier, linewidth=1, color='blue', label='T(t)')
plt.plot(tid, temperatur, 'x', color='red')
plt.grid()
plt.xlabel('t (sek)')
plt.ylabel('T (Grad celsius)')
plt.legend(loc='best')
plt.show()
```



2.17 Oppsummering

Matematiske operatorer

Operator	Utfører	Eksempel	Resultat
$+$	Addisjon	$5+2$	7
$-$	Subtraksjon	5-2	3
$*$	Multiplikasjon	$5*2$	10
/	Divisjon	5/2	2.5
//	Heltalldivisjon	13//3	4
**	Eksponent	2**3	8
$\div$	Modulo	$13 \div 4$	1

Datatyper

Syntaks	Utfører	Eksempel
str	streng (tekst)	'matematikk'
int	heltall	37
float	desimaltall	3.14
list	liste	['k', 1, 2, 'A']
set	mengde	$\{1, 3, 5\}$
bool	boolsk verdi	True, False
np.array	np-array-datastruktur	array([1, 2])

Å lese input fra brukeren og utskrift av tekst og variabel

Her tas et eksempel som viser hvordan man kan lese inn hele og reelle tall fra tastaturet, og skriver ut på skjermen, og hvordan man kan avrunde og formatere i utskriften:

```
print('Her leses inn variablene')
a = int(input("Skriv inn a (et heltall) her: "))
b = float(input("Skriv inn b (et reelt tall) her: "))
alder=input("Skriv inn alderen din:")
print(f''Du er {alder} år.'')
```

Andre tegn og syntaks

Tegn	betydning
==	lik
! =	ikke lik
>	mer enn
>=	mer eller lik (minst)
<	mindre enn
<=	mindre eller lik (høyst)

Importering av ulike pakker

```
import numpy as np
import matplotlib.pyplot as plt
```

Man kan enten importere hele biblioteket eller det man trenger fra biblioteket.

```
from numpy import *
from numpy import pi sin cos
```

Noen kommandoer

print ()	Skriv på skjerm
range(1,n+1)	Alle hele tall fra og med 1 til n
range(1,n+1,m)	Alle hele tall fra og med 1 til n med m steg lengde
range(1,n+1,m)	Alle hele tall fra og med 1 til n med m steg lengde
if ... elif ... else	Vilkår, if-test.
while	Løkke som kjører til testen er usann.
for	Løkke som kjører et på forhånd kjent antall ganger.
import matplotlib.pyplot as plt	Importerer selektivt pyplot-modulen til matplotlib som *plt*
plt.plot()	Tegner grafen til to lister.

Funksjoner

Funksjoner brukes til å dele opp programmer i mindre deler.

```
def funksjonsnavn(variabler):
  kodeblokk
```

- Første linje kalles *funksjonshode*, som består av `def`, fulgt av navnet på funksjonen i parenteser og variabler, som inngår i funksjonen et kolon :
- Resten av koden kalles kodeblokk, som hører til funksjonen som må tas med.

Python-filhåndtering

I Python er det ikke nødvendig å importere eksternt bibliotek for å lese og skrive filer. Python har en innebygd funksjon for å lage, skrive og lese filer.

Filkommandoer	Forklaring
f = open('filnavn')	Åpner en fil, returnerer filreferanse
f = open('filnavn','tilgang')	Åpner en fil, med spesifisert tilgang
	F.eks. 'w' åpner en fil for skriving (se neste side)
f.read()	Returnerer hele innholdet av filen
f.read(n)	Returnerer n karakterer av innholdet
f.readline()	Returnerer neste linje (før / n)
f.readlines()	Returnerer hele innholdet av fila som en liste
f.write(s)	Skriver strengen s til fil
f.writelines(liste)	Skriver innholdet av liste av strenger til fil
f.seek(offset fra hvor)	Forflytter filpekeren (index) i filen
f.tell()	Returnerer posisjon til filpekeren i filen
f.close()	Lukker filen

Tilgangstyper for open

Filkommandoer	Forklaring	Tidl. data	Hvis filen ikke finnes
'r'	Lesing	Bevares	FEIL
'w'	Skriving	Fjernes	Oppretter ny fil
'a'	Skriving (bakerst)	Bevares	Oppretter ny fil
'r+'	Lesing og skriving	Bevares	FEIL
'w+'	Lesing og skriving	Fjernes	Oppretter ny fil
'a+'	Lesing og skriving (helt bakerst)	Bevares	Oppretter ny fil

Kapittel 3

Programmeringsdidaktikk

For foreleseren

Den didaktiske programmeringskompetansen (hva, hvorfor og hvordan) spiller en viktig rolle i elevens læring. Det å kunne å ta i bruk algoritmisk tenking og programmering i klasserommet, krever faglig innsikt og forståelse for programmering i tillegg til den faglige kompetansen.

3.1 PRIMM-metoden for å tilrettelegge tekstprogrammering

Her har vi benyttet PRIMM-metoden for å tilrettelegge for læring av tekstprogrammering. PRIMM står for Predict-Run-Investigate-Modify-Make. PRIMM ble introdusert av Sue Sentance i 2017 og trekker sammen kjente effektive undervisningspraksiser i ett strukturert rammeverk.

- **Predict-fasen:** Forklar hva programmet utfører og hvilket problem programmet prøver å løse.
- **Run-fasen:** Kjør programmet for å lære hva programmet utfører, og eventuelt debugge (finne feilen) i programmet.
- **Inverstigate-fasen:** som bør varieres – gode øvelser som fremmer detaljene i programmering, og man lærer om syntaksene og kommandoene steg for steg. Oppgaver der det er lagt inn feil i programmmet og man skal finne feilen (feilsøking/debugging) i programmet.
- **Modify-fasen:** Man skal lære hva programmet utfører, og lære seg å gjøre nødvendige endringer for å tilpasse programmet til å løse et nytt problem. Man kan ha oppgaver, der man skal endre litt, og kan ha oppgaver der man skal endre flere deler for å få tilpasset til en ny problemstilling.
- **Make-fasen:** Når man har gått gjennom de første fire trinnene, kan det gis oppgaver der man kan låne kodebiter fra det originale programmet, men det vil ha en ny funksjon, kontekst eller problem som skal løses. Disse fem fasene er kanskje ikke i hver leksjon, avhengig av emnet. Og du kan sykle gjennom de tre, fire første fasene eller alle fem fasene flere ganger.

3.2 Hvordan man kan ta programmering i klasserommet og programmeringsdidaktikk

Det å lære hvordan man skal programmere i Python, og å lære realfaglig programmering, kan være nyttig for å kartlegge hvor i faget programmering kan innlemmes. Like viktig er det å lære hvordan man kan tilrettelegge programmering i undervisningstimene (programmeringsdidaktikk).

Det å dele erfaring med hverandre, kan være viktig for å få didaktiske ferdigheter nødvendig for å lære fra seg programmering. Her deles noen erfaringer med oversikten:



I neste del skal vi ta for oss følgende:

• **DEL 1:** Å integrere algoritmisk tenkning og programmering i egne fag

• **DEL 2:** Programmeringsppgaver (det studentene lærer på lavere nivå)

- **Algoritmisk tenkning**

- Oppsummering (algoritmisk tenkning)
- Programmeringsoppgaver

- **Tekstprogrammering (Python)**

- Oppsummering
- programmeringsoppgaver (Python)

• **DEL 3:** Programmeringsdidaktikk

- Eksempel for undervisningsopplegg i 10. trinn

- Eksempel for undervisningsopplegg i 1T, 1P, R2

3.3 DEL 1: Å integrere algoritmisk tenkning med enkle eksempler

Man kan gi elevene temaer de kan velge mellom å jobbe med der de skal jobbe med de 4 stegene til algoritmisk tenkning og sette opp et skjema som beskriver de 4 stegene for temaet.

Et enkelt eksempel fra hverdagslivet

La oss si at du må bestemme hva du skal gjøre med vennegjengen din. Hvis alle har ulike interesser, må du bestemme deg for:

- Hva kan dere gjøre? Hvor kan dere dra? Hvor lenge skal aktiviteten vare?
- Hvem vil gjøre hva?
- Hva har du tidligere gjort som har vært en suksess tidligere?
- Hvor mye penger har du og kostnaden til et av alternativene?
- Hvor mye tid har du å planlegge?

Fra denne informasjonen kan du og vennene dine lettere bestemme hvor du skal dra og hva du skal gjøre for å holde de fleste av vennene dine lykkelige. For slike problemer er det mulig å samle inn informasjon ved hjelp av et spørreskjema, analysere dataene for å finne den beste løsningen på problemet, både nå og også i et fremtidig scenario.

- Dersom man sender et brev i posten til en venn, hvordan kommer det frem?
- Du er håndballtrener og dere skal spille mot et annet håndballlag, hva gjør du for å velge taktikk?
- Dersom du er en fotballtrener og skal spille mot et annet fotballag som ofte angriper fra midtfelt, hvordan kan du ut fra motstandernes spillestill lage en taktikk? Hva om det viser seg at motlaget angriper omtrent bare med vingene sine, allerede 5-10 minutter etter kampstart?
- Tenk deg at bussen din går om 5 minutter og at tiden du bruker på å komme deg til busstoppet er 4 minutter. Hvis du løper, så rekker du bussen. Dersom du er uheldig og ikke rekker bussen, ender du opp med å måtte gå i 10 minutter for å så ta bybanen. Bussen bruker 20 minutter til skolen din, og bybanen bruker 30 minutter. Sett opp et skjema som beskriver situasjonen.

3.4 Integrere og etablere algoritmisk tenkning i egne fag

På ungdomstrinnet kan man benytte en rekke eksempler og oppgaver som gjør det mulig å ta i bruk de 4 stegene i algoritmisk tenkning. For eksempel:

- Bruk enkle eksempler som viser essensen til begrepene algoritmisk tenkning, modellering og programmering.
- En venn har kommet med en fantastisk, men likevel kompleks matlagingsoppskrift, og han/hun vil dele den med verden. For å gjøre dette må han/hun skrive et trinnvis detaljert sett med instruksjoner om hvordan du forbereder dette måltidet. En algoritmisk tenkningprosess er den ideelle måten å hjelpe noen på denne situasjonen. Først må personen fjerne

alle unødvendige detaljer. I dette tilfellet er det ikke nødvendig å vite hvor du skal kjøpe ingrediensene eller kostnadene knyttet til oppskriften (oppskriften må være universell, ikke begrenset av tilgjengeligheten av visse ressurser til et begrenset område). For å eliminere disse unødvendige detaljene, brukes *abstrahering* for å finne alle detaljene som bør unngås. For det andre bør instruksjonene angis så detaljert som mulig, og unngå ideer som er en selvfølge eller som sunn fornuft.

- Algoritmisk tenkning kan da brukes til å angi i detalje trinnene for å filetere en fisk eller å dele opp en kylling. Sett opp algoritmen.

  I videregåendeskole kan man etablere algoritmisk tankegang i både fellesfag og programfag. Dette krever ikke mer enn å forklare de fire trinnene og ta noen eksempler og de får noen gode eksempler, der kan praktiskere å sette opp de fem trinnene (dekomponering, mønstergjennkjenning, abstrahering, generalisering og vurdering). Her handler det også om å

- Bruke enkle eksempler fra faget ditt som viser essensen til begrepene algoritmisk tenkning, modellering og programmering.
- Bruke gode eksempler som fremmer begrepene mønstergjenkjenning og generalisering.
- Bruke spill som sjakk for å få elevene å tilegne seg logisk og algoritmisk tenkning.
- Gi enkle gode oppgaver som kan få elevene å lære seg stegene i algoritmisk tenkning, og lære seg å modellere, og oppgaver som tilrettelegge for å tilegne seg problemløsningsevnen.

3.5 Å integrere programmering i egne fag

Det er viktig å opparbeide en god arbeidsmetode for å tilrettelegge og implementere programmeringen i faget. Vi repeterer: Målet er ikke å programmere så mye som mulig, eller beherske flest mulig programmeringsspråk, men å tilegne seg forståelse for de ulike delene i programmeringsprosessen. Det blir her gitt et forslag som kan brukes som et utgangspunkt.

Gode og organiserte undervisningsopplegg for å lykkes
For å lykkes med programmering i skolen er det viktig

1. **Å legge opp til:**

   - en god struktur med klare mål som fremmer de viktigste læringsmålene, og sørger for at elevene har best mulig utbytte av undervisningen.

   - å vise sammenhenger mellom delemner i fag, og anvendelsen av faget i praktiske anvendelser.

2. **Å unngå løse og tilfeldige undervisingsopplegg**

Det finnes utallige mengder undervisningsopplegg tilgjengelige på internett, men veldig mange av dem er ikke laget med tanke på et pedagogisk utbytte i ditt fag. Skal en bruke tilfeldige opplegg, er det viktig at en som lærer ser nøye gjennom dem, og tilpasser dem til undervisningen og læringsmålene.

Å ha gode og vurderbare læringsmål
Ta utgangspunkt i kunnskapsmålene og lag forståelige læringsmål, slik at både du som lærer og

eleven vet hva som skal gjennomgås, læres og vurderes.

Stikkordsamling
Et godt arbeidsredskap er en god stikkordsamling, som du sammen med elevene fyller med forklaringer på fagbegrep og uttrykk. Denne bør være levende og oppdateres etter hvert som nye ord og uttrykk dukker opp, eller elevenes forståelse utvides slik at forklaringen må endres. Her har vi forsøkt å tilrettelegge med kontrollspørsmål og kontrolloppgaver som kan hjelpe deg å lage et eget undervisningsopplegg.

Prosjekt med programmeringsrelaterte arbeidsoppgaver
Ikke la selve programmeringen ta hele timen. Arbeid med forståelse er minst like viktig. Elevene trenger å forstå at metoder og prinsipper er generelle uansett verktøy en jobber i. Bruk tid til å ferdigstille prosjekter som kan lett deles i delprosjekter, dette kan gi elevene mestringsfølelse.

Egne notater, journal, blogg og evalueringsrapport
Oppmuntre og utfordre elevene å lage en journal/evalueringsrapport der de dokumenterer hva de har gjort. Det kan være skjermbilde av kode med kommentarer og forklaringer, refleksjoner over hva de har gjort, bilder av prosjekter de har arbeidet med, både underveis i prosessen og til slutt. En slik dokument hjelper læreren i en vurderingssituasjon, og elevene for å skape en dypere og bedre forståelse. Det gir også en god mulighet for læreren til å forbedre sitt undervisningsopplegg. Dette kan også bidra at elevene tilegner seg skrivekompetanse, og til at de utvikler kritisk tenkning, problemløsning og refleksjonsevne.

Refleksjon, kritisk tenkning og vurdering
Utfordre elevene til å reflektere over hvorfor ting skjer i koden og i ulike prosjekter. La dem lete etter sammenhenger og forklaringer. Refleksjoner får elevene til å tenke gjennom hva de gjør og hvorfor de gjør det, og gir dem en dypere forståelse av det de jobber med. Slike aktiviteter kan fremme prosessene for kritisk tenkning og vurdering av eget arbeid.

Analog koding
Analog koding er aktiviteter som viser koding uten datamaskin. Disse *hands-on*-aktivitetene kan være med på å lære elevene sammenhenger innen programmering. De kan lære metoder og prinsipper på en annen måte, som gjør det lettere å knytte dem til andre arbeidsoppgaver i faget. Det finnes en rekke aktiviteter på blant annet http://code.org og http://www.csunplugged.com.

3.6 Hvordan kan vi integrere algoritmisk tenkning/modellering/programmering i våre fag?

Her forsøker vi å skissere en plan for hvordan man kan integrere algoritmisk tenkning/modellering/programmering i ulike fag.

1. Hvor i faget kan algoritmisk tenkning/modellering/programmering integreres?

1. Ta utgangspunkt i læreplanen og søk etter hva som er kompetansemål/læringsmål for ditt fag og noter ned disse.
2. Sett opp en liste for de delene i ditt fag som kan belyses delvis eller fullstendig ved hjelp av komponentene ved problemløsning, algoritmisk tenkning, modellering og/eller programmering.
3. Det kan være nyttig å ha samarbeidspartnere fra andre læresteder eller skoler, hvor man kan dele og lære av hverandres synspunkter, tanker og ideer. Et slikt samarbeid danner også gunstige forutsetninger for utforming av gode undervisningsopplegg og prosjekter.

2. Viktig å lære begrepene:

1. Studentene skal få en kort opplæring om begrepene problemløsning/algoritmisk tenkning/-modellering/programmering.
   Hand out: Del ut et notat som beskriver disse med hjelpefigurer/hjelpediagrammer.
2. Man kan gjerne presentere for elevene hva begrepene læringsmål og kompetansemål er.
   Hand-out: Del ut et notat som beskriver disse.

3. Ferdigstille delbare prosjekter med klare læringsmål:

1. Lag eksempler/prosjekter som begynner med klare læringsmål som gis i begynnelsen av prosjektet. Man kan for eksempel begynne med: *Her skal vi lære ...*
2. Hvis prosjektet er delbar, kan vi med noen kommentarer lede elevene videre i prosjektet når de er ferdige med hver del. På denne måten kan elevene få mesteringsfølelse og blir interessert i å fortsette.
3. Man kan bruke disse prosjektene til å lære elevene nye emner også. Det kan da være viktig at de få tilgang til prosjektbeskrivelse på forhånd med et notat for hva elevene skal lære på forhånd (flipped-classroom).

4. Vurdere elevenes arbeid og evaluere prosjektet:

1. I slutten av prosjektet kan man sette opp noen kontrollspørsmål hvor elevene får mulighet til å reflektere over egen læringsprosess. Dette gjelder eksempelvis hva elevene selv mener de har lært og hva de behøver mer tid til, noe som kan føres opp i en journal, blogg eller et notat. Her er det viktig at elevene får en kort innføring i hvordan dette skal gjøres, og kanskje enda viktigere: hvorfor. Dette arbeidet kan være nyttig for å hjelpe elevene til å bygge en bro mellom læringsmål og kompetansemål. Det kan i tillegg hjelpe deg som lærer til å evaluere elevenes arbeid og læring. Dette arbeidet kan hjelpe elevene å bygge bro mellom læringsmål og kompetansemål. Dette kan også hjelpe deg å evaluere/måle elevenes arbeid/læring.
2. Når prosjektet er ferdig, kan du evaluere prosjektet og vurdere hvor det er muligheter for endring/forbedring.
3. Skriv en kort rapport om prosjektet.
4. Nå kan man tenke på nytt hvor i faget programmering kan integreres, og så utvide listen.

Hvordan lykkes med implementeringen av programmering?

Lærerne må få den opplæringen de trenger. For at opplæringen i programmering og algoritmisk tenkning skal lykkes og bidra til læring, forståelse og mestring, må temaet integreres i hele skoleløpet og flettes inn i matematikkfaget på matematikkfagets premisser. En grunnleggende forutsetning for at dette skal lykkes, er at alle matematikklærere i grunnskolen og videregående opplæring får en etter- og/eller videreutdanning som gir faglig trygghet. Ikke minst må lærerne selv ha tro på at algoritmisk tenkning, og programmering gir elevene grunnleggende kompetanse for å mestre det samfunnet de skal fungere i [24]. For å lykkes med implementeringen av programmering er det lærerens faglige og didaktiske kompetanse som er avgjørende (programmeringsdidaktikk). Det er derfor viktig å:

- Tilegne oss kunnskap om programmering
- Prøve å lære hvordan vi kan integrere programmering i våre fag
- Prøve å lære hvordan vi kan bruke programmering for å støtte opp under ulike emner, spesielt realfag, og formidle den til de andre (didaktiske ferdigheter i programmering eller programmeringsdidaktikk).

Viktig å ha tro på programmering og algoritmisk tenkning

Kjerneelementgruppen [24] peker på at matematikklærere i tillegg til å få en videreutdanning som gir faglig trygghet, må ha troen på at programmering og algoritmisk tenkning vil gi elevene en grunnleggende og nødvendig kompetanse, for at opplæringen skal kunne lykkes.

Som nevnt, det er mange gode argumenter for å innføre programmering i skolen, selv om programmering også fører med seg en del utfordringer for lærere og elever. Programmering er en kompetanse som anses som fremtidsrettet og kan bidra til dybdelæring,kompetanse i algoritmisk tenkning, og kunne påvirke elevers sosiale og emosjonelle kompetanser. Det er felles enighet om innføringen vil medføre stofftrengsel, og for noen elever kan det bli for mange digitale verktøy å forholde seg til. Læreren og lærerens profesjonsfaglige digitale kompetanse trekkes frem som de viktigste forutsetningene for at implementeringen skal lykkes. I tillegg vil eksamen ha betydning for i hvor stor grad verktøyet vil tas bruk i undervisningen, på grunn av eksamenens sterke *washbackeffekt* [7].

Oppsummering - viktige begrep

Problemløsning: handler om å forstå problemet, dele opp problemstillingen, analysere fakta og opplysningene, prøve å finne en tilpasset løsningsmetode, løse problemet, reflektere, vurdere hvor gyldig løsningen er, og evaluere løsningsstrategien, og hele problemløsningsprosessen,.

Algoritmisk tenkning: er en problemløsningsmetode. Algoritmisk tenkning innebærer å tilnærme seg problemer på en systematisk måte, både når vi formulerer hva det er vi ønsker å løse, og når vi foreslår mulige løsninger.

Modellering: handler om aktivitet og prosessen for forenkling av virkeligheten som brukes for å beskrive tilstander og egenskaper som det er vanskelig eller umulig å definere/måle direkte. Modellering derier seg om å kunne omformulere en problemstilling fra virkeligheten, til et matematisk problem, løse dette problemet og så vurdere gyldigheten og begrensningen til løsningen i den opprinnelige situasjonen.

Matematisk modellering: handler om å kunne omformulere en problemstilling fra virkeligheten, til et matematisk problem, løse dette problemet og så vurdere gyldigheten og begrensningen til løsningen i den opprinnelige situasjonen.

Programmering: er prosessen med å lage et sett med instruksjoner som forteller en datamaskin hvordan du skal utføre en oppgave.

Koding: er bare den delen av programmeringen som handler om å fortelle en datamaskin hva den skal gjøre. Koding i skolen dreier seg om at elevene lærer å løse problemer og samarbeide på kreative måter. Koding er brukt i mange sammenhenger i skolen som et begrep som omfatter programmeringsaktiviteter.

Realfaglig programmering og matematisk modellering handler om problemløsing, algoritmer, logisk tenkning og argumentasjon. Matematikk utgjør hovedtyngden av opplæringen, men programmering kommer inn i flere fag. Kunnskap om numeriske metoder og realfaglig programmering åpner for å utforske fagene mer i dybden, og tilrettelegger for både bredde- og dybdelæring. I programmering får elever bruk for kunnskap om numeriske metoder, som er matematiske løsningsteknikker tilpasset datamaskinen.

3.7 DEL 2: Programmeringsoppgaver (skoletrinnet)

Denne delen skal hjelpe foreleseren å få kjennskap til Python-programmering på lavere trinn, og kan hjelpe studentene å repetere det de burde ha lært på skoletrinnet. Forskjellige lærere vektlegger ulikt og det er fint for elevene å ha tilgang til programmeringsoppgavene. Håper at disse oppgavene kan hjelpe deg å få oversikt over grunnleggende elementer i Python-programmering.

Algoritmisk tenkning - Oppsummering

Algoritmisk tankegang er en problemløsningsmetode, og handler om tanke prosessen, konsepter og fremgangsmåter fra et problem til man har funnet en løsning. Seks elementer/steg/trinn som inngår i algoritmisk tenkning [22]:

- Logikk - analysere og forutse (hypoteser)
- Algoritmer - lage steg-for-steg-beskrivelser og regler
- Dekomponering - bryte ned og dele problemet opp i mindre, mer håndterbare deler.
- Mønstergjenkjenning - gjenkjenne og utnytte likheter
- Abstrahere/Abstraksjon - fjerne unødvendige detaljer
- Evaluering - vurdere resultat og prosessen

Det å integrere algoritmisk tenkning i et problem handler om å sette opp de seks trinnene (analysere, algoritmisering, dekomponering, mønstergjennkjenning, abstrahering, og evaluering) for problemet. For et matematisk problem handler også om å finne en løsningsstrategi og prøver å løse problemet.

Programmeringsoppgaver (algoritmisk tenkning)

Oppg. 3.7.1:
Her tar vi noen enkle oppgaver som kan hjelpe oss å hvordan vi kan integrere algoritmisk tenkning i forskjellige fag.

a) Skriv kort om de 6 stegene til algoritmisk tenkning

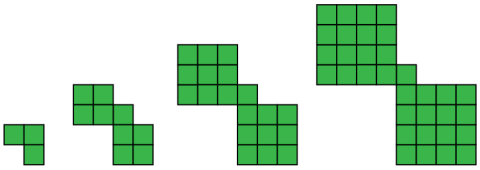
b) Sett opp matoppskriften for å lage hvete boller med de stegene i algoritmisk tenkning.

c) Du bor i Bergen, og du skal reise i påsken til familien i Stavanger. Prøv å bruke de stegene i algoritmisk tankegangen for å arrangere reisen fra der du bor til du kommer hjem til familien (bestille billett buss/tog/fly, ta bybane buss/drosje til Flesland, ...)

d) Gitt summen:

$$1 + 2 + 3 + \cdots + 47 + 48 + 49$$

Oppg. 3.7.2: Se figurene under:

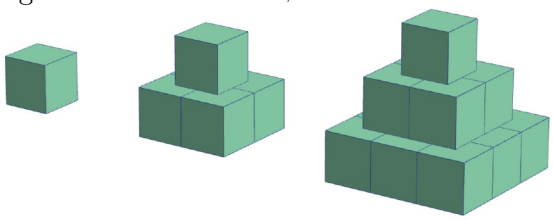


a) Hvor mange små kvadrater du totalt trenger for å lage de 10 første figurene.

b) Lag en algoritme som du kan bruke til å bestemme hvor mange små kvadrater du totalt trenger for å lage de 100, og de n første figurene.

Oppg. 3.7.3: Eksamen 1P-våren 2022

Nedenfor ser du tre figurer. Figurene er satt sammen av små klosser. Roar vil fortsette å lage figurer etter samme mønster.



Figur 1 Figur 2 Figur 3

Tenk deg at du skal fortsette å lage figurer etter samme mønster.

a) Hvor mange klosser trenger han for å lage figur 5?

b) **Ekstra oppgavedel:** Lag en algoritme (algoritmisk tekning) som du kan bruke til å bestemme hvor mange små klosser du totalt trenger for å lage figur nr. 7.

c) Roar har 10 000 klosser. Han vil starte med den minste figuren og lage én figur i hver størrelse.
Hvor mange figurer kan han lage? Hvor mange klosser vil han ha igjen når han har laget figurene?

d) **Ekstra oppgavedel:** Lag et program med utgangspunkt i algoritmen og svarer på forrige oppgavedel. Programmet skal også beregne og skrive ut hvor mange klosser du trenger for et gitt antall klosser som leses av brukeren.

3.8 Python- litt oppsummering

Matematiske operatorer

Operator	Utfører	Eksempel	Resultat
+	Addisjon	$5 + 2$	7
−	Subtraksjon	5-2	3
*	Multiplikasjon	$5 * 2$	10
/	Divisjon	5/2	2.5
//	Heltalldivisjon	13//3	4
**	Eksponent	2**3	8
÷	Modulo	$13 \div 4$	1

Datatyper

Syntaks	Utfører	Eksempel
str	streng (tekst)	'matematikk'
int	heltall	37
float	desimaltall	3.14
list	liste	['k', 1, 2, 'A']
set	mengde	$\{1, 3, 5\}$
bool	boolsk verdi	True, False
np.array	np-array-datastruktur	array([1, 2])

Å lese input fra brukeren og utskrift av tekst og variabel

Her tas et eksempel som viser hvordan man kan lese inn hele og reelle tall fra tastaturet, og skriver ut på skjermen, og hvordan man kan avrunde og formatere i utskriften:

```
print('Her leses inn variablene')
a = int(input("Skriv inn a (et helttall) her: "))
b = float(input("Skriv inn b (et reelt tall) her: "))
```

Andre tegn og sytaks

Tegn	betydning
==	lik
!=	ikke lik
>	mer enn
>=	mer eller lik (minst)
<	mindre enn
<=	mindre eller lik (høyst)

Gjentagelser, hvilkår, betingelser (løkker)

En løkke brukes for beregninger eller handlinger som skal gjentas flere ganger.

- En **if/elif/else**-løkke brukes når vi ønsker å utføre en blokk med kode når den gitte tilstanden er sant eller usant.

```
a = float(input("Skriv inn a (et reelt tall) her: "))
if a> 0:
    print("a er positivt")

elif a<0:
    print("a er negativt")

else:
    print("a er lik null")
```

- **while**-løkker brukes i tilfeller hvor man kan si at løkka skal utføres så lenge noe er sant.

```
i=0
while i < 3:
    print("i =", i)
    i += 1
```

- **for**-løkken kan brukes til å gjenta ei kodeblokk et kjent /begrenset antall ganger eller gå gjennom en sekvens, element for element:

```
n=2
for i in range(1,n+1):
        a=2*i+1
        print(a)
```

I situasjoner der vi ikke helt vet hvor lenge vi vi kjøre en løkke, while kan brukes. En for-løkke brukes når vi vet på forhånd hvor mange ganger løkken kan gå.

Importering av pakker/numeriske biblioteker

```
import numpy as np
import matplotlib.pyplot as plt
```

Man kan enten importere hele biblioteket eller det man trenger fra biblioteket.

```
from numpy import *
from numpy import pi sin cos
```

I de nye matematikk lærebøkene brukes *pylab*-biblioteket som har mange innbygte funksjoner.

```
import pylab
```

Noen foretrekker numpy på denne måten:

```
import numpy as np
```

der man for eksempel for å bruke π skal ha $np.pi$ og sin-funksjonen skal skrive $np.sin(x)$. Fordi blant annet har man da et bedre oversikt hvor i programmet har brukt pakken.

Noen kommandoer

print ()	Skriv på skjerm
range(1,n+1)	alle hele tall fra og med 1 til n
range(1,n+1,m)	alle hele tall fra og med 1 til n med m steg lengde
range(1,n+1,m)	alle hele tall fra og med 1 til n med m steg lengde
if ... elif ... else	Vilkår, if-test.
while	Løkke som kjører til testen er usann.
for	Løkke som kjører et på forhånd kjent antall ganger.
import matplotlib.pyplot as plt	Importer selektivt pyplot-modulen til matplotlib som *plt*
plt.plot()	Tegner grafen til to lister.

Funksjoner

Funksjoner brukes til å dele opp programmer i mindre deler.

```
def funksjonsnavn(variabler):
  kodeblokk
```

- Første linje kalles *funksjonshode* består av `def`, fulgt av navnet på funksjonen parenteser og variable som inngår funksjonen et kolon :.
- Resten av koden kalles kodeblokk som hører til funksjonen som må tas med.

For eksempel:

```
'Funksjon som regner ut volumet av en kjegle'
'med radius r  og hoyde h som gitt av inputparametrene.'

import numpy as np

def Vkjegle(r,h):
    volum = (np.pi*(r**2)*h)/3
    return volum

r=3
h=4
Volum=Vkjegle(r,h)
print(f'Volumet til en kjegle med radius {r} og høyden {h} er {Volum:.3
    f}')
```

Plotting av grafer til funksjoner

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-3,3, num = 101)

def f(x):
    f = x**3-3*x
    return f

plt.plot(x,f(x), '--')
plt.grid()
plt.xlabel('x')
plt.ylabel('y')
plt.legend(['f(x)'])
plt.savefig('MinPythonFigMappe/eksplot.png')
```

Python-filhåndtering

I Python er det ikke nødvendig å importere eksternt bibliotek for å lese og skrive filer. Python har en innebygd funksjon for å lage, skrive og lese filer.

Filkommandoer	Forklaring
f = open('filnavn')	Åpner en fil, returnerer fil referanse
f = open('filnavn','tilgang')	Åpner en fil, med spesifisert tilgang
	F.eks. 'w' åpner en fil for skriving (se neste side)
f.read()	Returnerer hele innholdet av filen
f.read(n)	Returnerer n karakterer av innholdet
f.readline()	Returnerer neste linje (før / n)
f.readlines()	Returnerer hele innholdet av fila som en liste
f.write(s)	Skriver strengen s til fil
f.writelines(liste)	Skriver innholdet av liste av strenger til fil
f.seek(offset fra hvor)	Forflytter filpekeren (index) i filen
f.tell()	Returnerer posisjon til filpekeren i filen
f.close()	Lukker filen

Tilgangstyper for open

Filkommandoer	Forklaring	Tidl. data	Hvis filen ikke finnes
'r'	Lesing	Bevares	FEIL
'w'	Skriving	Fjernes	Oppretter ny fil
'a'	Skriving (bakerst)	Bevares	Oppretter ny fil
'r+'	Lesing og skriving	Bevares	FEIL
'w+'	Lesing og skriving	Fjernes	Oppretter ny fil
'a+'	Lesing og skriving (helt bakerst)	Bevares	Oppretter ny fil

3.9 Oppsummering - Enkle programmeringsoppgaver

Oppg. 3.9.1: Hva blir resultat av følgende programsetninger/programkoder:

a)
```
print('2+.5')
```

b)
```
print(2+.5)
```

c)
```
print('2+3)
```

d)
```
Print('2+3')
```

e)
```
print('5+7')
print()
```

f)
```
Print('5+7')
```

g)
```
print('5+7')
print()
```

h)
```
18/5 + 18//5
```

i)
```
for i in range(1,4):
    a= 5*i
    print(a, end=" ")
```

j)
```
for i in range(0,8,2):
    print(i, end=" ")
```

k)
```
for i in range(5,1,-1):
    print(i, end=" ")
```

l)
```
i = 1
while i < 5:
  print(i, end = ' ' )
  if i == 3:
    break
  i += 1
```

Oppg. 3.9.2:

a) Hva utfører følgende program?

```
import numpy as np

r= float(input('Skriv inn radiusen til en sirkel: '))

def A(r):
    return np.pi*r**2

print(A(r))
```

Hva blir svaret hvis man skriver som svar 1 når man kjører programmet?

b) Skriv et program som leser fra brukeren radius og høyden til en rett sylinder og regner ut volumet, og printer ut resultatet.

Oppg. 3.9.3: Hva utfører følgende program?

```
km = float(input("Hva er reiselengden (KM): "))
p= int(input("Antall passasjerer: "))
# kostnad regning
if km<5:
    kostnad=250
else:
    kostnad = 200 + km*10
# Hvis antall passasjerer er mer enn 4, da må bestille maxi-taxi som koster 1.25 ganger mer.
    if p >= 5:
        kostnad = kostnad * 1.25
# Den totale kostnaden
print(f'Reisekostnaden for {p} passasjerer er Kr.{kostnad:.2f}')
```

Kan du skrive et lignende program for et lignende problem der et mobilabonnement der vedkommende betaler:

- 100 kroner hvis vedkommende sender mindre enn 500 meldinger pr. mnd.
- ellers betaler vedkommende fastpris 50 kr og 0.1 kr per melding, og hvis sende
- Hvis man sender mer enn 1000 meldinger pr. måned, får man fastpris 150 kr.

man kan ringe gratis til alle i dette abbonementet.

Hvor mye betaler man per måned hvis vedkommende sender x antall meldinger?

Oppg. 3.9.4:

a) Forklar kort hva følgende program utfører:

```
import numpy as np
import matplotlib.pyplot as plt

xstart = -2
xslutt = 2
steglengde = 0.01

def f(x):
    return x**3 + 3*x - 1

x = xstart
y = f(x)

while x <= xslutt:
    x += steglengde
    y = f(x)
    print(f'{x:.3f} {y:.3f}')

x = np.arange(xstart, xslutt, steglengde)
y = f(x)

plt.plot(x,y, '--')
plt.grid()
plt.xlabel('x')
plt.ylabel('y')
plt.legend(['f(x)'])
plot.show()
```

b) Utvid programmet ditt til å la brukeren velge minste og største verdi for x og trinnene, i tabellen.

c) Kunne vi bruke `x=np.linspace(xstart, xslutt, 401)` istedenfor `x = np.arange(xstart, xslutt, steglengde)`?

Oppg. 3.9.5: Forklar kort hva programmet utfører:

```
import statistics as stat

data = [121,115,131,134,127,138]

MD=stat.mean(data)
STD=stat.stdev(data)

print(f'Standard avviket er {STD}')
print(f'Middelverdien er {MD} ')
```

3.10 Programmeringsppgaver (Python)

Gjentagelser, hvilkår, betingelser (løkker)

Oppg. 3.10.1: a) Skriv hva resultatet til følgende programkode når man kjører dem:
Programkode 1:

```
x = 1
while x < 10:
    x += 1
    print(x,' ',end='')
```

Programkode 2:

```
x = 1
while x < 10:
    print(x,' ',end='')
    x += 1
```

b) Hvordan kunne vi få samme resultat med for-løkken. Bestem m og n.
Programkode 1:

```
m = ...
n = ...
print()
for i in range(m,n):
    i+=1
    print(i,' ',end='')
```

Programkode 2:

```
m = ...
n = ...
for i in range(m,n):
    print(i,' ',end='')
    i+=1
```

Oppg. 3.10.2: Lån og sparing (10. trinn) En låner 5000 kr. fra en kredittkonto som tilbyr 2.5% månedelig fastrente. Vi vil bestemme hvor lenge det tar før lånebeløpet har blitt 15 000 kr. Følgende programkode prøver å løse denne problemstillingen:

```
vekstfaktor = 1.025
Antallmåneder = 0
y= 5000
sluttverdi=15000

while y < sluttverdi:
    y = y*vekstfaktor
    Antallmåneder += 1

print("Etter", Antallmåneder, "måneder blir lånebeløpe", y)
```

a) Forklar hvorfor *while*-løkke ble brukt her og ikke *for*-løkke?

b) Kjør programmet og skriv hva resultatet blir.

c) Hva må oppdateres i programmet hvis man skal spare opp 10 000 med å sette inn 5000 i en sparekonto som gir 1.5 % hver måned?

d) Hvordan kan du sjekke svarene du fikk i del a)? Hvordan kan dette forklares ved regning?

Oppg. 3.10.3: Prosentendring (10. trinn) Erfaring viser at prisen til en nybil synker med 18% pr. år eksponentielt. Prisen til en nybil er 350 000.
Vi vil bestemme prisen når den er minst halvert.
Følgende programkode prøver å løse denne problemstillingen:

```
1
2 vekstfaktor = ...
3 startverdi =  ...
4 verdi= startverdi
5 i=0
6
7 while verdi >= startverdi/2:
8     print(i,verdi)
9     verdi = verdi*vekstfaktor
10    i+=1
11
12 print(i, verdi)
13 print("Etter", i, "år er verdien mer enn halvert og prisen er", verdi)
```

a) Hva skal vekstfaktor og verdi være? Kjør programmet og skriv hva resultatet blir.

b) Hvordan kan du sjekke svarene du fikk i del a)? Hvordan kan dette forklares ved regning?

Oppg. 3.10.4: Serielån (10. trinn) En låner 2 500 000 kr. med et serielån med en nedbetalingstid på 20 år og med årlige terminar. Renta er 2.5 % per år. Vi vil bestemme årlige terminer.
Følgende programkode prøver å løse problemstillingen:

```
1 L = 2500000
2 t= 20
3 p= 0.025
4
5 avdrag =L/t
6
7 for i in range(1,t+1):
8     rente = L*p
9     terminbeløp = avdrag + rente
10    L = L - avdrag
11
12    print(i,int(terminbeløp))
```

a) Forklar linje for linje hva programmet utfører.

b) Kjør programmet og skriv hva resultatet blir.

c) Hva må oppdateres i programmet hvis rentesatsen går opp til 3%?

Oppg. 3.10.5: Annuitetslån (1T, R2)
Med et annuitetslån betaler du det samme beløpet hver måned, gitt at renten er uforandret. Anta at 100 000 kr. skal betales tilbake med rentefot 5 % i 8 år med 2 termin er pr. år (16 terminer).

a) Sett opp en geometrisk rekke som bestemmer lånebeløpet L ved hjelp av terminbeløpet T.

b) Bestem terminbeløpet T.

c) Her ønsker vi å generalisere problemstillingen. Anta lånebeløpet L skal betales tilbake med rentefot $p\,\%$ i t år med n terminer pr. år.
Sett opp et uttrykk for L gitt terminbeløpet T og vekstfaktor a ($a = 1 + p/100$).

d) Bestem terminbeløpet T uttrykt ved L og p.

e) Sett opp et program som beregner terminbeløpet T i forrige oppgavedel.

Oppg. 3.10.6: Skriv en programkode som spør brukeren hvilken farge brukeren ser i trafikklyset (rød, oransje, grønn). Brukeren får tilbakemelding etter hvilken farge man velger.
Hvis man velger *rød*, får man tilbakemelding *STOP*.
Hvis man velger *oransje*, får man tilbakemelding *Klar til å kjøre*.
Hvis man velger *grønn*, får man tilbakemelding *Kjør!*.

Oppg. 3.10.7:

Skriv hva følgende programkode utfører:

```
tall = int(input("Skriv inn et partall: "))

if tall%2 == 1:
    print("Tallet er ikke et partall!")
else:
    sum = 0
    i=0
    while(i <= tall):
        sum = sum + i
        i = i+2

print(sum)
```

Hvis man skriver 4 som svar for spørsmålet man får ved å kjøre programmet, hva blir resultatet?

Skriv en programkode som gjør dette for oddetall.
Hvis man skriver 5 som svar for spørsmålet man får ved å kjøre programmet, hva blir resultatet?

Oppg. 3.10.8: En drosjeselskap har følgende regler for å regne reisekostnader:
- Hvis reiselengden er mindre enn 5 km., er fast pris 250 kr.
- Fast start pris er 200 kr. For hver km. regnes 10 kr.
- Hvis antall passasjerer er mer enn 4, må bestilles maxi taxi som koster 1,25 ganger mer.

Skriv en programkode som leser inn reiselengden og antall passasjerer fra brukeren, og regner ut reisekostnaden og skriver ut resultatet.

Oppg. 3.10.9: Skriv hva resultatet av følgende programkode blir:

a)
```
i=0
for i in range(2):
    print('A')
for i in range(3):
  print('B')
```

b)
```
for i in range(1):
    print('A')
    for i in range(2):
        print('B')
```

c)
```
i=0
for i in range(2):
    print('A')
    for i in range(4):
        print('B')
```

Oppg. 3.10.10: Skriv en programkode som gir følgende som resultat når man kjører den:

a)
```
A
A
A
B
B
```

b)
```
A
B
B
A
B
B
A
B
B
```

Oppg. 3.10.11: Skriv hva resultatet blir når man kjører følgende programkode:

a)
```
for x in [30, 40]:
    print()
    for y in [1, 2, 3]:
        print(x+y, end='\n')
```

b)
```

for x in [30, 40]:
    print()
    for y in [1, 2, 3]:
        print(x+y, end=' ')
```

Oppg. 3.10.12: Skriv hva resultatet av følgende programkode blir:

a)
```
r = 3
for i in range(1, r + 1):
    print(i,i+1, end=' ')
    print()
```

b)
```
r = 6
for i in range(1, r + 1, 2):
    print(i,i+1, end=' ')
    print()
```

c)

```
r = 4
for i in range(1, r + 1, 1):
    for j in range(1, i + 1):
        print(j, end=' ')
    print()
```

d)

```
for x in [100, 200]:
 for y in [1, 2, 3]:
     print(x+y, end=' ')
```

Importering av pakker/numeriske biblioteker

Oppg. 3.10.13: a) Hva må rettes/tilføyes i følgende program for at programkoden fungerer:

```
def Vsylinder(r,h):
  V=pi*r**2*h
  return V

V=Vsylinder(3,5)

print(V)
```

b) Skriv en programkode som beregner volumet til en kule med radius r, og skriver ut volumet til en kule med radius 4.

Funksjoner

Oppg. 3.10.14:

a) Skriv hva følgende programkode utfører og hva resultatet blir:

```
def Atrekant(g,h):
  A=(g*h)/2
  return A

A=Atrekant(5,6)
print(A)
```

b) Skriv en programkode som bergner arealet til et trapes med sidelengdene a, b og h, og regner ut arealet til et trapes der $a = 3$, $b = 5$ og $h = 4$.

Hint: Arealet til et trapes med sidelengdene a, b og h: $A = \frac{(a+b)}{2}h$.

Oppg. 3.10.15: La f være funksjonen gitt ved $f(x) = -x^3 + x + 2$. Skriv en programkode som setter opp x-verdiene, mellom -2 og 3 med steglengde 0.25, en funksjonssyntaks for $f(x)$ som beregner y-verdiene til f, og lagrer x- og y-verdiene i to lister.

Oppg. 3.10.16: I denne oppgaven skal vi se på kokektiden til egg.

a) Erfaring viser at vannskoketid ved høyden h kan beregnes ved

$$KT = 100 - 0.0032 * h$$

Skriv en programkode som setter opp en funksjon som beregner vannskoketid ved høyden h. Bruk programmet til å bestemme koketiden i Everest, der høyden er 8848, og programmet skal skrive ut svaret.

b) Koketiden til egg som

$$tid = A * log((2 * (Tvann - T0))/(Tvann - Tplomme))$$

der $A = 3.75$.

Oppg. 3.10.17: Forklar kort hva følgende programkoden utfører.

```
import random

def main() :
    t1 = random.randint(1,100)
    t2 = random.randint(1,100)

    print("Hva blir summen:",t1,"+",t2)
    print()
    attempt = 1
    print()
    svar = int(input("svar: "))

    while svar != t1 + t2:
        if svar < t1 + t2:
            print("Feil! Rett svar er mer enn ",svar,".")
            print()
            attempt += 1
        elif svar>t1+t2:
            print("Feil! Rett svar er mindre enn ",svar,".")
            print()
            attempt += 1
        else:
            attempt += 1
            svar = int(input("Svar: "))
            print("Bra! Rett svar er: " + str(svar) + "." + "\n Du
   klarte det etter " + str(attempt) + " forsøk")

main()
```

Fil- og databehandling

Oppg. 3.10.18: Opprett en text fil *fil.txt*. Skriv i filen *Vi ses snart!* og lukk filen.
Skriv ut innholdet i filen (åpne filen og skriv ut det som leses fra filen).
Kjør programkoden.
Hvis vi ikke lukker filen, hva skjer hvis vi kjører programmet flere ganger?
Hvordan kan vi slette alt innhold i filen før vi begynne å kjøre programmeet?

Oppg. 3.10.19: I denne oppgaven skal vi lese data (tid og temperatur) fra filen Tekopp-temperatur.csv". Filen begynner med noen kommentarer og deretter to søyler med *Tid* og *Temperatur* som navn:

Tid (sek)	Temperatur (^oC)
0	81.50
310	62.50
620	45.70
910	36.80
1220	29.40
1810	25.30
2550	24.20
3100	23.30
3620	22.50

Følgende fil ble brukt: *http : //home.hib.no/ansatte/ahas/BK/Tekopp/Tekopp−temperatur.csv*

a) Hva må rettes/tilføyes i følgende program for at programkoden fungerer:

```
df = pd.read_csv('http://home.hib.no/ansatte/ahas/BK/Tekopp/Tekopp-
    temperatur.csv',sep=';', decimal='.',comment='#',encoding='latin
    -1')
tid = df['Tid'].tolist()
temperatur = df['Temperatur'].tolist()
tstart = 0
Tstart = temperatur[0]
tslutt =3700
deltat = 0.5
To = 22 # Rom temperatur
k = .0015 # proporsjonalitets koeff.
tverdier = []
Tverdier = []
t = tstart
T= Tstart
while t <= tslutt:
    tverdier.append(t)
    Tverdier.append(T)
    t = t + deltat
    deltaT = -deltat * k * (T - To)
    T = T + deltaT
plt.plot(tverdier,Tverdier,linewidth=1,color='blue',label='T(t)')
plt.plot(tid, temperatur, 'x', color='red')
plt.grid()
plt.xlabel('t (sek)')
plt.ylabel('T (Grad celsius)')
plt.legend(loc='best')
plt.show()
```

b) Foklar hva programmet utfører generelt. Forklar hva programmet utfører linje for linje.

Litt statistikk

Oppg. 3.10.20: En programkode er gitt ved:

```
import numpy as np

def mean(data):
  n = len(data)
  mean = sum(data) / n
  return mean

def std(data):
  n = len(data)
  mean = sum(data) / n
  deviations = [(x - mean) ** 2 for x in data]
  STD = np.sqrt(sum(deviations) / n)
  return STD

data = [168,172,167,175,176,178]
MD=mean(data)
STD=std(data)
print(f'Standard avviket er {STD}')
print(f'Middelverdien er {MD} ')
```

a) Forklar kort hva programmet utfører.

b) Hva må vi endre i programmet for at programkoden regner ut middelverdi og emprisk standardavvik (frihetsgrad $n-1$) til datasettet:

$$Liste = [12, 15, 21, 11, 17, 19, 23, 27, 18, 25]$$

Oppg. 3.10.21:

```
import statistics as stat

data = [168,172,167,175,176,178]

MD=stat.mean(data)
STD=stat.stdev(data)

print(f'Standard avviket er {STD}')
print(f'Middelverdien er {MD} ')
```

Forklar kort hva programmet utfører.

Litt statistikk med import random

Oppg. 3.10.22: En programkode er gitt som::

```
import random
i= 0
sum = 0
øye1 = 0
øye2 = 0
øye3 = 0
øye4 = 0
```

```
øye5 = 0
øye6 = 0

antallkast = int(input("Hvor mange ganger vil du kaste terningen?"))

while i < antallkast:
    utfall= random.randint(1,6)
    if utfall== 1:
        øye1 = øye1 + 1
    elif utfall== 2:
        øye2 = øye2 + 1
    elif utfall== 3:
        øye3 = øye3 + 1
    elif utfall== 4:
        øye4 = øye4 + 1
    elif utfall== 5:
        øye5 = øye5 + 1
    elif utfall== 6:
        øye6 = øye6 + 1
    sum = sum + utfall
    i = i + 1

print("Gjennomsnittsverdi av oyne for", antallkast, "ganger kast av
    terningen", (sum * 1.00) / (utfall * 1.00), ":)")
print("Med følgende resultater:\n")
print("En:  ", øye1, "ganger")
print("To:  ", øye2, "ganger")
print("Tre:", øye3, "ganger")
print("Fire: ", øye4, "ganger")
print("Fem: ", øye5, "ganger")
print("Seks: ", øye6, "ganger")
```

a) Forklar kort hva programmet utfører.

b) Skriv en programkode som simulerer et myntkast et antall ganger som blir lest fra brukreren og teller opp hvor mange ganger hvert av utfallene (mynt og kron) forekommer, og skriver ut dette som resultat.

c) Skriv en programkode som simulerer en stein-saks-papir lek, et antall ganger som blir lest fra brukreren og teller opp hvor mange ganger hvert av utfallene (stein, saks og papir) forekommer, og skriver ut dette som resultat.

3.11 DEL 3: Programmeringsdidaktik (å ta programmeringen til klasserommet)

Vekstfaktor og anvendelser (10. trinn)

Oversikt

Her er et eksempel hvordan vi kan sette opp undervisningsmateriell som gir oversikt over kompetansemål, litt teori og programmeringsoppgaver:

- Læringsmål
- Litt teori
- Oppgaver

Matematikk - Læringsmål (kompetansemål - 10. trinn)

Mål for opplæringen er at eleven skal kunne

- *Utforske og generalisere multiplikasjon av polynomer algebraisk og geometrisk*
- *Utforske og sammenligne egenskaper ved ulike funksjoner ved å bruke digitale verktøy, lage, løse og forklare ligningssett knyttet til praktiske situasjoner*
- *Regne ut stigningstallet til en lineær funksjon og bruke det til å forklare begrepene endring per enhet og gjennomsnittsfart*
- *Utforske sammenhengen mellom konstant prosentvis endring, vekstfaktor og eksponentialfunksjoner*
- *Hente ut og tolke relevant informasjon fra tekster om kjøp og salg og ulike typer lån og bruke det til å formulere og løse problemer*
- *Planlegge, utføre og presentere et utforskende arbeid knyttet til personlig økonomi*
- *Bruke funksjoner i modellering og argumentere for framgangsmåter, og resultater modellere situasjoner knyttet til reelle datasett, presentere resultatene, og argumentere for at modellene er gyldige*
- *Utforske matematiske egenskaper og sammenhenger ved å bruke programmering*

Oppsummering. **Vekstfaktor og prosentfaktor**

En verdi endrer seg fra x_0 til x_1.
1) Absolutt endring: $x_1 - x_0$,

2) Relativ endring: $\dfrac{x_1 - x_0}{x_0}$

3) Vekstfaktor: $\dfrac{x_1}{x_0}$

4) Prosentfaktor: $\dfrac{x_1}{x_0} - 1$

En verdi endrer seg med p prosent pr. tidsenhet (år, mnd., $\cdots$). Vekstfaktor er da $1 + \dfrac{p}{100}$.

der p kalles rentefot, prosentsats eller rentesats.

Verdi etter endring = Verdi før endring · Vekstfaktor

rentesats (p)	Prosentfaktor	Vekstfaktor
En verdi vokser med 25 %	0,25	1,25
En verdi synker med 25 %	0,75	0,75
En verdi vokser med 100 %	1	2

Hvis en verdi K_0 har endret seg med $p\%$ eksponentielt i løpet av en tidsenhet, er verdien etter n tidsenheter gitt ved:

$$K(n) = K_0(1 + \frac{p}{100})^n$$

Serielån

Med et serielån betaler du mindre hver måned etter hvert som du betaler ned på lånet. Renteandelen synker når tiden går, mens avdragsdelen er den samme.

Terminbeløp= Avdrag + Rente = Avdrag + Lånebeløp · Rentesats

Oppgaver
Her kan man bruke oppgave 3.10.2-3.10.4 og kan gjerne endre teksten til oppgavene for nye situasjoner. ■

Programmeringsdidaktikk. **Prosjektoppgave: Modellering av tekopp-temperatur (1T, 1P, 2P, R2)**

Vi ønsker å studere endringen i tekopp temperatur i løpet av en time (60 min).

Læringsmål (Kompetansemål etter 10. trinn)

– *Utforske sammenhengen mellom konstant prosentvis endring, vekstfaktor og eksponentialfunksjoner*
– *Bruke funksjoner i modellering og argumentere for framgangsmåter og resultater*
– *Modellere situasjoner knyttet til reelle datasett, presentere resultatene og argumentere for at modellene er gyldige*
– *Utforske matematiske egenskaper og sammenhenger ved å bruke programmering*

Læringsmål (Kompetansemål etter matematikk 1P)

– *Tolke og bruke funksjoner i matematisk modellering og problemløsing.*

Læringsmål (Kompetansemål etter matematikk 1T)

– *Modellere situasjoner knyttet til ulike temaer, drøfte, presentere og forklare resultatene og argumentere for om modellene er gyldige*

Læringsmål (Kompetansemål etter matematikk 2P)

– *Vurdere valg knyttet til personlig økonomi og reflektere over konsekvenser av å ta opp lån og å bruke kredittkort*

Læringsmål (Kompetansemål etter matematikk R2)

– *Gi eksempler på ulike situasjoner som kan modelleres ved å bruke ulike matematiske funksjoner, og modellere og analysere slike situasjoner ved å bruke reelle datasett*

• **Læringsutbytte**
– *Elevene skal forstå hvordan kurven som beskriver temperaturfallet avtar raskest i starten for så å nærme seg temperaturen i rommet som er en asymptote til kurven.*

– *Elevene skal se at lineær kurvetilpasning ikke kan dekke behovet, spesielt i de første minuttene*

Programmeringsdidaktikk

Teori (det tas litt teori på tavla før eksperimentet begynner):

Newtons avkjølingslov: Dette forsøket prøver å lage en matematisk modell som beskriver hvordan temperaturen endres i et objekt (en kopp med vann) som kjøles ned av omgivelsene. Funksjonsuttrykket:

$$y = f(t) = at + b$$

representerer en lineær funksjon som kan beskrive lineær sammenheng mellom to størrelser. Funksjonsuttrykket:

$$y = f(x) = c + ae^b t$$

der $b < 0$.

representerer en eksponentialfunksjon som ved start (t=0) har verdien $y = c + a$, kan senke mot c når tiden (t) går uendelig. Den kan beskrive sammenheng mellom to størrelser (y og t), der y synker eksponentiell ved tiden t.

Utstyr:
- 10 eller flere kopper (samme type)
- Vannkoker
- En vekt
- En klokke
- Ett eller flere termometer(e)

Tidsbruk: Forarbeid: 15-20 min og elevene jobber ca. 60 min, 10-15 min refleksjon over detaljene i prosjektet med åpen samtale og eventuelt kontroll spørsmål.

Gjennomføring:

Kok opp tilstrekkelig vann til å fylle opp alle koppene. Sett koppene på vekta etter tur og fyll opp med like mye av det varme vannet. Mål temperaturen med jevne mellomrom i løpet av en time (3600 sek), for eksempel etter 5 min (300 sek), 10 min (600 sek), ... og noter resultatet i tabellen.

Tid (sek)	Temp (Celsius)

For eksempel (data hentet fra time-temp.pdf (southampton.ac.uk))

Tid (sek)	Temp (Celsius)
0	81.50
310	62.50
620	45.70
910	36.80
1220	29.40
1810	25.30
2550	24.20
3100	23.30
3620	22.50

Eksempel 3.11.3: I dette eksempelet skal vi nå se på filen 'tekopp-temperatur.csv'. Hver linje i filen er en temperatur-måling i en te-kopp i løpet av en time (3600 sekunder) som er plassert i et rom med temperatur 22 Grad celsius. Her bruker vi Newtonsavkjølingslov for å studere temperaturendringen i vannkoppen.

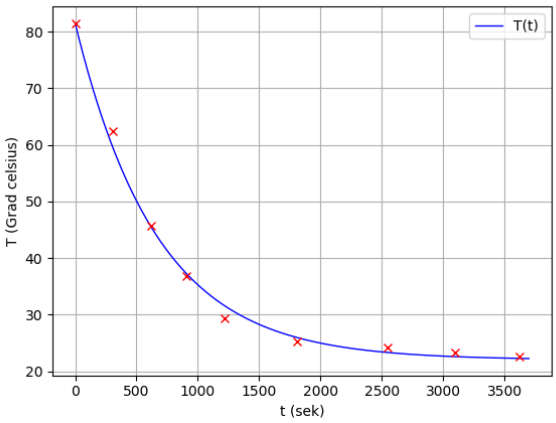
$$\frac{dT}{dt} = k(T - T_o)$$

der $T(t)$ ((målt i $^\circ$ celsius) er vanntemperatur i vannkoppen ved tiden t (målt i sekunder).

```
import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv('http://home.hvl.no/ansatte/ahas/MP/tekopp-temp.csv
    ',sep=';', decimal='.', comment='#',encoding='latin-1')

tid = df['Tid'].tolist()
temperatur = df['Temperatur'].tolist()
tstart = 0
Tstart = temperatur[0]
tslutt =3700
deltat = 0.5
To = 22 # Rom temperatur
k = .0015 # proporsjonalitets koeff.
tverdier = []
Tverdier = []
t = tstart
T= Tstart
while t <= tslutt:
    tverdier.append(t)
    Tverdier.append(T)
    t = t + deltat
    deltaT = -deltat * k * (T - To)
    T = T + deltaT
plt.plot(tverdier,Tverdier, linewidth=1,color='blue',label='T(t)')
plt.plot(tid, temperatur, 'x', color='red')
plt.grid()
plt.xlabel('t (sek)')
plt.ylabel('T (Grad celsius)')
plt.legend(loc='best')
plt.show()
```


T(t)
T (Grad celsius)
t (sek)
80
70
60
50
40
30
20
0
500
1000
1500
2000
2500
3000
3500

Programmeringsdidaktikk. **Å søke løsning for ligningen $f(x) = 0$**

Først vil vi si noe om en ligning har en løsning på et intervall (eksistens), og om den har bare og bare én løsning på intervallet (entydighet)

Eksistens og entydighet

1. Når har funksjonen, $y = f(x)$, et nullpunkt nær et punkt eller ligger på et intervall $[a, b]$?
   Eller: Når har ligningen $f(x) = 0$ en løsning nær et punkt eller ligger på et intervall?

   (a) Funksjonen er kontinuerlig om punktet (på intervallet).

   (b) Funksjonen er monoton på intervallet $[a, b]$.

   (c) Funksjonen er kontinuerlig om punktet (på intervallet) og har forskjellige fortegn på endepunktene, det vil si $f(a) \cdot f(b) < 0$

2. Når har funksjonen, $y = f(x)$, *ett* nullpunkt nær et punkt eller ligger på et intervall $[a, b]$?

   Eller: Når har ligningen $f(x) = 0$ *bare* en løsning nær et punkt eller ligger på et intervall?

   (a) Funksjonen er kontinuerlig om punktet (på intervallet).

   (b) Funksjonen er monoton på intervallet $[a, b]$.

   (c) Funksjonen er kontinuerlig om punktet (på intervallet), $f(a) \cdot f(b) < 0$ og er monoton på intervallet.

En funksjon er *kontinuerlig* på et intervall: Grafen er sammenhengende på intervallet.
En funksjon er *monoton* på et intervall: Grafen bare stiger eller bare synker på intervallet.

Eksempel 3.11.5: Har ligningen $x^3 = 5$ é løsning på intervallet $[1\,,\,2]$?
Svar: Alle de to kravene er tilfredsstilt:
1) Eksistens: $f(x) = x^3 - 5$ er kontinuerlig på intervallet $[1\,,\,2]$, og $f(1) = -4$ og $f(2) = 3$ og dermed $f(1) \cdot f(2) < 0$, det vil si grafen skjærer x-aksen minst en gang,
2) $f(x) = x^3 - 5$ er monoton på intervallet $[1\,,\,2]$, altså den bare stiger på intervallet $(f'(x) = 3x^2 > 0)$

3.12 Halveringsmetoden for å løse $f(x) = 0$

Programmeringsdidaktikk. Halveringsmetoden er en enkel iterativ metode for å finne et nullpunkt til en funksjon, det vil si å finne en løsning for ligningen $f(x) = 0$.

Når kan vi bruke halveringsmetoden?
For å bruke halveringsmetoden:
1) $y = f(x)$ skal være kontinuerlig på intervallet $[a, b]$ der løsningen ligger,
2) $f(a) \cdot f(b) < 0$, det vil si må vi kjenne to punkt a og b som er slik at $f(a)$ og $f(b)$ har ulikt fortegn. ■

Halveringsmetode - Algoritme

Algoritmen starter med å finne midtpunktet $x_m = m$ mellom a og b. Hvis f(m) og f(a) har ulikt fortegn, så søker vi videre i intervallet $[a, m]$. Hvis ikke fortsetter vi med intervallet $[m, b]$. Slik kan vi fortsette helt til den tilnærmede løsningen tilfredsstiller et estimeringskrav.

Problemstillingen tas i en klasse, og en elev kommer med følgende algoritme:

```
# Definerer funksjonen f(x) = x^3 -5
def f(x):
        return x**3 -5

n=201
x = linspace(-2,2,n)
# Lager en liste  x-verdier jevnt fordelt i intervallet [-2,2]

y = f(x)
# Lager en liste med de 200 tilhørende y-verdiene

for i in range(0,n-1):
    if y[i]*y[i+1]<=0:
        nullpkt = (x[i]+x[i+1])/2
print("Jeg har funnet et nullpunkt. x=", nullpkt)

>>Jeg har funnet et nullpunkt. x= 1.7085427135678393
```

Algoritmen sjekker i listen og nærmere seg mot løsning slik at to nærliggende punkt skal ha forskjellige fortegn.
Hva er det som mangler på denne elev-algoritmen (forrige side)? Eller hvordan kan vi effektivisere algorimen?

Fremgangsmåte (algoritme):

Finn midtpunktet $m = (a + b)/2$
Hvis $f(m) = 0$, så er m løsningen.
Ellers hvis $f(m) \neq 0$:
Hvis $f(a) \cdot f(m) < 0$, ligger roten mellom a og m. Så vi går igjen for a og m.
Eller hvis $f(b) \cdot f(m) < 0$ så ligger roten mellom b og m. Så vi gjentar b og m.

```
def f(x):
    return x**3-5

a = 1
b = 2
presisjon = 0.1

m = (a+b)/2   #Finn m, midtpunktet mellom a og b.
d=b-a

while abs(d)>presisjon:
    if f(a)*f(m) <0:  #Hvis f(a) og f(m) har motsatt fortegn:
        b = m     #Flytter b til midten
    elif f(b)*f(m)<0: #Hvis f(a) og f(m) har motsatt fortegn:
        a = m     #Flytter a til midten
    elif f(m)==0:
        break
    else:
        print("De ligger på sammeside")
    #print(f 'f({a}) og f({b}) ligger på samme side av x-aksen')
        break
    m=(a+b)/2

print("Nullpunktet er ca. x=:",m)
```

Ved å kjøre programmet får vi:

```
Nullpunktet er ca. x = 1.71044921875
```

Programmet leser inn endepunktene

```
def f(x):
  return x**3-5
print("Vi skal bestemme nullpunktet til f(x)=x^3 - 5 naer 1")
a = float(input("Skriv a: "))
b = float(input("Skriv b: "))

d = b - a #Finn avstanden mellom a og b

xm = (a+b)/2   #Finn m, midtpunktet mellom a og b.

fa = f(a) #Regn ut f(a)
fb = f(b) #Regn ut f(b)

fm = f(xm)  # Regn ut funksjonsverdi i midtpunktet f(xm)

while abs(d) > 0.001:

    if fm*fa > 0:  #Hvis fm og fa har samme fortegn:
        a = xm     #Flytter a til midten
        fa = fa
    else:
        b = xm     #Flytter b til midten
        fb = fb

# Sett opp et nytt midtpunkt og reknar ut f-verdi:

    xm = (a+b)/2
    fm = f(xm)
    d = b - a

print("Nullpunktet er ca.:",xm)
```

Ved å kjøre programmet får vi:

```
Vi skal bestemme nullpunktet til f(x)=x^3 - 5 naer 1
Skriv a: 0
Skriv b: 2
Nullpunktet er ca.: 1.71044921875
```

3.13 Elev-metoder for å løse ligningen $f(x) = 0$

Det kan godt tenkes at elevene kommer med spennende forslag som er gitt nedenfor. Prøv å reflektere over metodene og om de har noen ulemper.

Metode 1: Deltax-metoden:

Ta utgangspunkt i et av endepunktene, og velg en stegslengde *deltax* for å nærme seg mot løsningen:

```
def f(x):
    return x**3 - 5
a = 1
b = 2
n = 0 # Antall iterasjoner

Nøyaktighet = .001
deltax = .01
while deltax > Nøyaktighet:
    if f(a) * f(a + deltax) < 0:
        deltax = deltax / 10
    else:
        a = a + deltax
        n = n+1
print(a)
print("Antall iterasjoner:",n)
```

Ved å kjøre programmet får vi:

```
1.7000000000000006
Antall iterasjoner: 70
```

Metode 2: Tilfeldig søk-metoden

Her brukes tilfeldig søk gjennom intervallet. I denne algoritmen:

- Begynner man fra et av endepunktene (Bestex) og regner funksjonsverdien (Bestey=f(Bestex)).
- Det plukkes tilfeldig en x fra intervallet $[a, b]$, regner ut $y = f(x)$ og hvis: $abs(y) < abs(Bestey)$, erstattes $Bestex$ med x og $Bestey$ med y. Dette gjentas i en løkke så lenge $abs(y) < abs(Bestey)$.

```
import random

def f(x):
    return x**3-5

a = 1
b = 2
N = 1000
n= 0
Bestey = f(a)
Bestex = a
for i in range(N):
    x = random.uniform(a, b)
    y = f(x)
    if abs(y) < abs(Bestey):
        Beste_y = y
        Beste_x = x
        n= n+1
print(Bestex)
print("Antall iterasjoner:",n)
```

Ved å kjøre programmet flere ganger får vi naturligvis forskjellige verdier med forskjellige antall iterasjoner:

```
1.7095742320073466
Antall iterasjoner: 11

1.7102058920572012
Antall iterasjoner: 9
```

Fasit

Fasit 2.4.1:

a)
```
print('Denne setningen skrives ut!')
```

b) P i print skal være liten bokstav. Mangler apostrof i enden til setningen.

Fasit 2.4.2:

a) 5+7

b) 12

c) SyntaxError. Legg merke til at det mangler en apastrof etter 7.

d) Print skal være print.

e)
```
5+7
```

Fasit 2.4.3:

a)
```
> 3*5+4/5
15.8
```

b)
```
> 2**3+3**2+1.25/4
17.25
```

c) Resultatet til 18%4 blir 2 (restleddet i divisjonen 18 på 4).

Fasit 2.4.4:

a) Skriver og kjører i console:
```
> 5*8/2
20.0
```

b) Skriver og kjører i konsollvinduet:
```
> 7*10
70.0
```

c) Skriver og kjører i konsollvinduet:
```
>6*(8+10)/2
54.0
```

Fasit 2.6.1:

a)

```
m=7
n=1
m=m+3  # eller m+=3
n=n*5   # eller n*=5
p= m*n
print('Produktet blir da',p)
```

Svaret blir da 30.

b)

```
a=5
b='TALL'
a=a*2
b=b* 3

print(a)
print(b)
```

Utskriften er da:

```
10
TALLTALLTALL
```

Fasit 2.6.2:

```
l = 5
b = 4
h = 3
volum=l*b*h
overflate = 2*l*b + 2*b*h + 2*l*h
print(f"Volume er {volum}.")
print(f"Overflaten er {overflate}.")
```

Ved å kjøre programmet blir svaret:

```
Volume er 60.
Overflaten er 94.
```

Fasit 2.6.3:

```
h= 3.5
g = 9.81
v0 = 20
t = 5

y= h - 0.5*g*t**2+v0*t

print('y er lik',y)
```

Fasit 2.7.1:

```
a = float(input("Skriv inn a (et reelt tall) her: "))
b= 3
c= 4
d = (1/3)*a*b+c**2

print('d er lik',d)
```

Fasit 2.7.2:

```
r = float(input("Skriv inn radiusen til en sylinder (et reelt positivt
    tall) her: "))
h= float(input("Skriv inn radiusen til en sylinder (et reelt positivt
    tall) her: "))
pi=3.14
V = pi*r**2*h

print('Volumet til sylinderen er lik',V)
```

Fasit 2.8.1:

```
import numpy as np

x=0.5
y = np.sin(np.pi*x)

print('y er lik',y)
```

Fasit 2.8.2:

a) Programkode:

```
import numpy as np
svar1=np.cos(2)+np.tan(1.5)
print(svar1)
```

b) Programkode:

```
import numpy as np
svar2=np.exp(4)+np.log10(2)+np.log(0.1)
print(svar2)
```

Fasit 2.8.3:

```
import pylab

x=0.5
y = sin(pi*x)

print('y er lik',y)
```

Fasit 2.10.1:

Hvis man kjører programmet, blir man spurt `Skriv et tall`, og når man skriver hvis tallet er negativt, blir den erstattet med det tallet med positiv fortegn. Til slutt skriver programmet ut `Absluttverdien til a er x`, der x er et positivt tall uansett hva input a var. Resultatet blir blir `Absluttverdien til -7  er 7`.

Fasit 2.10.2:

a) Programkode:

```
tall = int(input("Skriv inn et tall: "))
if tall > 0:
    print(f"Tallet {tall} er positivt.")
elif tall < 0:
```

```
    print(f"Tallet {tall} er negativt.")
else:
    print(f"Tallet {tall} er null.")
```

b) Programkode:

```
tall = int(input("Skriv inn et positivt tall: "))

if tall ==0 or tall <0:
    print(f"{tall} er ikke positivt, skriv et positivt tall.")
elif tall%2 == 0:
    print(f"Tallet {tall} er jevntall.")
else:
     print(f"Tallet {tall} er oddetall.")
```

c) Programkode:

```
Tall1 = float(input("Skriv inn et tall: "))
Tall2 = float(input("Skriv inn et tall: "))

if Tall1*Tall2 < 0:
    print(f"Tallene {Tall1} og {Tall2} har motsatt fortegn.")
elif Tall1*Tall2 > 0:
    print(f"Tallene {Tall1} og {Tall2} har samme fortegn.")
else:
    print(f"Et av tallene {Tall1} eller {Tall2} er null.")
```

Fasit 2.10.3:
Programkoden spør hva summen $49 + 35$, og når brukeren svarer, programkoden svarer om svaret ikke er rett (*Regn igjen!*) eller om det er rett (*Du klarte det!*).

Fasit 2.10.4:

```
innlest = input("Hva er 52+19?")
tall = int(innlest)

if tall != 71:
    while tall != 71:
        innlest = input("Feil. Regn igjen!")
        tall = int(innlest)
    else:
        print("Du klarte det!")
```

Fasit 2.10.5:

```
tall = int(input("Skriv inn et oddetall: "))

if tall%2 != 0:
    print("Tallet er ikke et oddetall!")
else:
    sum = 0
    i = 1
    while(i <= tall):
        sum = sum + i
        i = i + 2

print(sum)
```

```
Skriv inn et oddetall: 5
9
```

Fasit 2.10.6:

```
x0= 1
b = 1024
x = 0
x1 = x0

while x1 != b:
    x += 1
    x1*= 4

print(f"Svaret er {x}.")
```

```
Svaret er 5
```

Fasit 2.10.7:

a) Resultatet blir:

```
A
B
B
B
A
B
B
B
```

b) Resultatet blir:

```
51 52 53 61 62 63
```

c) Resultatet blir:

```
51 52 53
61 62 63
```

Fasit 2.10.8:

a) Resultatet blir:

```
0 1 2 3 4 5 6 7 8 9
```

b)

```
1 2 3 4 5 6 7 8 9
```

c)

```
0 2 4 6 8
```

d)

```
10 9 8 7 6 5 4 3 2
```

Fasit 2.10.9:

```
svar = ""

while svar != "nei":
    svar = input("Hei! Vil du fortsette programmet? ")

print("Du tastet nei, programmet avsluttes")
```

Fasit 2.10.10:

```
a1 = 3
k = 2
n=5
Sum=0
for i in range(n):
    an=a1
    an = a1 * k
    Sum+=an

print(f'Summen til {n}  ledd i en geometrisk rekke med a1 {a1}  og
    kvotient {k} er {Sum} ')
```

Fasit 2.10.11:
Programkoden lister opp for hvert tall fra og med til og med 9, tallet fra (fra og med) 1 til (til og med) 4.

Fasit 2.10.12:

```
a1 = 5
k = 3
n=7
Sum=0
for i in range(n):
    an=a1
    an = a1+k
    Sum+=an

print(f'Summen til {n}  ledd i en geometrisk rekke med a1 {a1}  og
    kvotient {k} er {Sum} ')
```

Fasit 2.10.13:

a) Programkoden lister opp for hvert tall fra og med til 1 og med 2, tallet fra (fra og med) 2 til (til og med) 6.

```
1 2
1 4
1 6
2 2
2 4
2 6
```

b) `range(5)` sender inn $0, 1, 2, 3$ og 4 og resultatet blir for i in range(5):

```
02438
```

c) `while i < 4` sender inn $1, 2, 3$ og 4 og resultatet blir

```
1
2
3
```

while-delen erstattes med `for i in range(1,4):`

d)

```
1
2
3
4
i er ikke lenger mindre enn 5
```

e)

```
1
1 2
1 2 3
1 2 3 4
```

f)

```
o
o o
o o o
* * * *
* * *
* *
*
```

g)

```
n=4;
for i in range(n):
    for j in range(i):
        print ('* ', end="")
    print('')

for i in range(n,0,-1):
    for j in range(i):
        print('o ', end="")
    print('')
```

Fasit 2.10.14:

a) Programkode:

```
n = 4
k = 4
for i in range(n+1,1,-1):
    for j in range(1,k-i+3,1):
        print(j,end=' ')
    print()
```

b) Programkode:

```
n = 4
k = 4
for i in range(1,n+1,1):
    for j in range(k-i+1,0,-1):
```

```
        print(j,end=' ')
    print()
```

c) Programkode:

```
n = 4
k = 4
for i in range(n+1,1,-1):
    for j in range(k-i+2,0,-1):
        print(j,end=' ')
    print()
```

d) Programkode:

```
n = 4
k = 4
for i in range(n+2,0,-1):
    for j in range(1,k-i+3,1):
        print(j,end=' ')
    print()
```

Fasit 2.11.1:

```
man får Skalarproduktet av a og b er 1.0 . Legg merke til $-1\cdot 0+3\
    cdot 1 + 0 +\cdot 5 = 1$
```

Fasit 2.11.2:

a)

```
import numpy as np

def A(r):
    return np.pi*r**2

print(A(5))
```

b)

```
import numpy as np

r= float(input('Skriv inn radiusen til en sirkel: '))

def A(r):
    return np.pi*r**2

print(A(r))
```

Fasit 2.11.3:

a) Programkode 1 (0.25 ligger i konvergensintervall til s_1. Programkode 1 og 2 (1.75 ligger i konvergensintervall for begge) Programkode 2 (2.25 ligger i konvergensintervall til s_2

b) Ved å synke estimeringsnøyaktighet e i programkodene.

Fasit 2.11.4:

Programkode:

```python
import numpy as np

def A(a,b,c):
    s=1/2*(a+b+c)
    A=np.sqrt(s*(s-a)*(s-b)*(s-c))
    return A

a = 3
b = 4
c = 5

Areal=A(a,b,c)
print(f'Arealet til en trekant med sidelengdene {a}, {b} og {c} er {Areal}')
print(Areal)
```

Fasit 2.11.5:

Programkode:

```python

def KE(m,v):
    """
Denne funksjonen bestemmer kinetisk enegi for en partikkel gitt:
Input:
    masse (kg)
    hastighet(m/s)

Output:
   Partikkelens kinetisk energi (J)
    """
    KE = 0.5 * m * v** 2
    return KE

m= 0.0001
v=8
E=KE(m, v)
print(f'Kinetisk energi for en partikkel med massen {m} og hastighet {v} m/s er {E} J')
```

Fasit 2.11.6:

```
5
15
```

Fasit 2.12.1:

a)

- `plt.plot(x,f,'g--')` tegner grafen til funksjonen og tar inn x-verdiene definert i `x = np.linspace()`, de tilhørende funksjonsverdiene, og deretter `'b--'` skifter utseendet på grafen til en stiplet i stedet for en sammenhengende graf med grønnfarge. Flere argumenter kan angis i `plt.plot()` for å spesifisere detaljene til grafen.
- Funksjonen `np.linspace(start, end, num = N+1)` deler intervallet $[start, end]$ inn i N like deler med $N + 1$ jevnt fordelte punkter.
- `plt.yticks([-15,-5,5,15])` spesifiserer tallverdiene på y-aksen.

- `plt.grid()` legger på et gitter bak grafen.
- `plt.xlabel()` og `plt.ylabel()` definerer navn på aksene.
- `plt.legend()` legger til en tekstboks for å beskrive hva grafen heter. `loc='best'`): plasserer tekstboksen der det passer best.
- `plt.show` tegner diagrammet i konsoll-vinduet.
- `plt.savefig()` Denne funksjonen gjør at vi kan lagre figuren direkte når vi kjører koden. Her må vi legge til riktig path inne i en streng, slik at bildet blir plassert i den mappen vi ønsker.

b)

Fasit 2.12.2:

a)

```
import matplotlib.pyplot as plt
import numpy as np

xstart = -2
xslutt = 2
steglengde = 0.01

def f(x):
    return x**3 + 6*x - 3

x = xstart
y = f(x)

while x <= xslutt:
    x += steglengde
    y = f(x)
    print(f'{x:.3f} {y:.3f}')

x = np.arange(xstart, xslutt, steglengde)
y = f(x)

plt.plot(x,y)
```

b) Programkode:

```
xstart = float(input('Skriv inn nedre grense for x: '))
xslutt = float(input('Skriv inn øvre grense for x: '))
steglengde  = float(input('Skriv inn steglengden: '))

def f(x):
    return x**3 + 6*x - 3

x = xstart
y = f(x)

while x <= xslutt:
    x += steglengde
    y = f(x)
    print(f'{x:.3f} {y:.3f}')
```

Fasit 2.12.3:

Programkode:

```
1
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  # Her skal vi definere en funksjon, og deretter plotte den
6
7  # Deler inn x-aksen i 100 deler (101 punkt)
8  x = np.linspace(-2*np.pi,2*np.pi, num=101)
9
10 def f(x):
11     f= np.sin(2*x-np.pi)
12     return f
13
14 plt.plot(x, f(x), color='green')
15
16 def g(x):
17     g= np.cos(2*x+np.pi)
18     return g
19
20 plt.plot(x, g(x), color='blue')
21
22 def h(x):
23     h= np.cos(2*x+np.pi/2)
24     return h
25
26 plt.plot(x, h(x), color='red')
27
28 plt.show()
```

f og h representerer samme funksjon.

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  c= 60
5  a=120
6  T= 365
7  k =2
8  d= 182.5
9
10
11 # Deler inn x-aksen i 1000 deler (1001 punkt)
12 x = np.linspace(0,T, num=1001)
13
14 def f(x):
15     f= c+a*np.sin((2*np.pi*k/T)*x+ d)
16     return f
17
18 plt.plot(x, f(x), color='blue')
19 plt.grid()
20 plt.xlabel('x ')
21 plt.ylabel('y')
22
23 def g(x):
24     g= c+a*np.cos((2*np.pi*k/T)*x+d)
25     return g
```

```
26
27 plt.plot(x, g(x), color='red')
28
29 plt.legend(loc='best')
30 plt.legend(['f(x)','g(x)'])
31
32 plt.show()
```

Fasit 2.13.1:

a) [1, 2, 3, 4, 5, 2, 4, 6, 8, 10]

b) A=[1,2,3,4,5]

c) Programkode:

```
1 liste = [1, 2, 3, 4, 5, 6]
2
3 produkt = 1
4
5 for t in liste:
6     produkt *= t
7
8 print(produkt)
```

Fasit 2.13.2:

```
1  def f(x):
2      return -.2*x**2+1
3
4  xstart = -1
5  xslutt = 2
6  xsteg = .25
7
8  xverdier = []
9  yverdier = []
10
11 x = xstart
12 y = f(x)
13
14 while x <= xslutt:
15     xverdier.append(x)
16     yverdier.append(y)
17     x = x + xsteg
18     y = f(x)
19
20 print(xverdier)
21 print(yverdier)
```

Fasit 2.13.3:

```
1 def f(x):
2     return -x**3+x+2
3
4 xstart = -2
5 xslutt = 3
6 xsteg = .25
```

```
7
8  xverdier = []
9  yverdier = []
10
11 x = xstart
12 y = f(x)
13
14 while x <= xslutt:
15     xverdier.append(x)
16     yverdier.append(y)
17     x = x + xsteg
18     y = f(x)
19
20 print(xverdier)
21 print(yverdier)
```

Fasit 3.7.1:

a) Resultatet blir:

- Logikk - analysere og forutse (hypoteser)
- Algoritmer - lage steg-for-steg-beskrivelser og regler
- Dekomponering - bryte ned og dele problemet opp i mindre, mer håndterbare deler.
- Mønstergjenkjenning - gjenkjenne og utnytte likheter
- Abstrahere/Abstraksjon - fjerne unødvendige detaljer
- Evaluering - vurdere resultat og prosessen

b) Prøv å analysere problemet, og skriv steg-for-steg som kan gjøres for å lage boller. De 6 stegene er:

- Logikk - analysere og forutse (hypoteser)
- Algoritmer - lage steg-for-steg-beskrivelser og regler
- Dekomponering - bryte ned og dele problemet opp i mindre, mer håndterbare deler.
- Mønstergjenkjenning - gjenkjenne og utnytte likheter
- Abstrahere/Abstraksjon - fjerne unødvendige detaljer
- Evaluering - vurdere resultat og prosessen

c) Prøv å analysere problemet, og skriv steg-for-steg som kan gjøres for å lage en reiseplan fra der du bor til hjem til familien. De 6 stegene er:

- Logikk - analysere og forutse (hypoteser)
- Algoritmer - lage steg-for-steg-beskrivelser og regler
- Dekomponering - bryte ned og dele problemet opp i mindre, mer håndterbare deler.
- Mønstergjenkjenning - gjenkjenne og utnytte likheter
- Abstrahere/Abstraksjon - fjerne unødvendige detaljer
- Evaluering - vurdere resultat og prosessen

d) Bruk algoritmisk tankegangen for å bestemme summen uten å bruke kakultator/datamaskin. Problem: Vi skal bestemme summen:

$$S = 1 + 2 + 3 + \cdots + 47 + 48 + 49 + 50$$

1) Dekomponering/Nedbrytning: Deler problemet i mange delproblemer:

$$1 + 2 = 3$$
$$3 + 4 = 7$$
$$5 + 6 = 11$$
$$\vdots$$
$$49 + 50 = 99$$

Dette fører ikke til noe som jeg gjør problemstillingen lettere.
Men hvis vi deler opp problemet slik at vi summerer det første og det siste tallet, det neste tallet og det nest siste tallet, og videre:

$$1 + 50 = 51$$
$$2 + 49 = 51$$
$$3 + 48 = 51$$
$$\vdots$$
$$25 + 26 = 51$$

2) Mønstre - gjenkjenne og utnytte likheter: Vi ser at vi har 50 ganger av samme sumverdi og dermed summen blir

$$S = 51 * 50 = 2550$$

For å regn ut $S = 50 * 51$, kunne vi tenke slik også: $S = (50 + 1) \cdot 50 = 50 \cdot 50 + 50 = 2500 + 50 = 2550$.
3) Abstraksjon (fjerne unødvendige detaljer): Vi kan bare tenke at

$$1 + 50$$

og dette gjentar seg 50 (halvparten av antall ledd fra 1 til 50) ganger og dermed:

$$S = (1 + 50) \cdot \frac{50}{2}$$

4) Algoritme/Generalisering:

$$1 + 2 + 3 + \cdots + (n - 1) + n = (1 + n) \cdot \frac{n}{2}$$

Mer generalisering:

$$1 + 2 + 3 + \cdots + a_{n-1} + a_n = (a_1 + a_n) \cdot \frac{n}{2}$$

der $d = a_n - a_{n-1}$ differansen mellom to påfølgende ledd er konstant (aritmetisk rekke).

Kommentar: Her kunne tenkes også slikt:

$$S = 1 + 2 + 3 + \cdots + 48 + 49 + 50$$
$$S = 50 + 49 + 48 + \cdots + 3 + 2 + 1$$

Legge sammen disse og får da:

$$2S = 51 + 51 + 51 + \cdots + 51 + 51 + 51$$

Dermed;

$$2S = 51 \cdot 50$$
$$S = 51 \cdot \frac{50}{2} = 2550$$

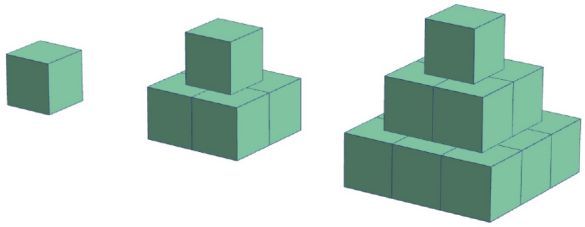
Fasit 3.7.2:

a) Figur 1: $2 * 1^2 + 1 = 3$
Figur 2: $2 * 2^2 + 1 = 9$
Figur 3: $2 * 3^2 + 1 = 19$
Figur 4: $2 * 4^2 + 1 = 33$
$\vdots$
Figur 10: $2 * 10^2 + 1 = 201$

b) Fra del a) ser vi at figur 100 skal ha $2 * 100^2 + 1 = 20001$ små kvadrater og dermed i figur n, antall små kvadrater blir $a(n) = 2 * n^2 + 1 = 2n^2 + 1$ der $n = 1, 2, 3, \cdots$.

Fasit 3.7.3:

a) Figur nr. 5: $1 + 4 + 9 + \cdots + \cdots = 1^2 + 2^2 + 3^2 + 4^2 + 5^2 = 1 + 4 + 9 + 16 + 25 = 55$.



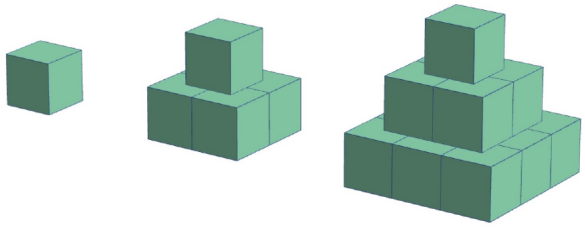
Figur 1 Figur 2 Figur 3

Figur 1: 1
Figur 1: $1 + 4 = 1^2 + 2^2$
Figur 3: $1 + 4 + 9 = 1^2 + 2^2 + 3^2$
$\vdots$ Figur 4: $1+4+9+? = 1^2+2^2+3^2+4^2 = 30$ Figur 5: $1+4+9+?+? = 1^2+2^2+3^2+4^2+5^2 = 55$

b) Implementering av algoritmisk tenkning

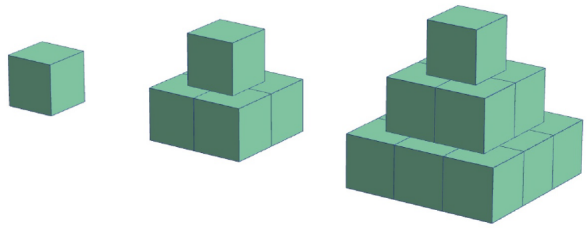
- Logikk - analysere
  Her ser vi at figuren har forskjellige struktur når figuren utvikler-utvider seg.
- Algoritmer - lage steg-for-steg-beskrivelser og regler - Her ser at vi kan finne regel for hver del., og kan deretter sette dem sammen.
- Dekomponering - bryte ned og dele problemet opp i mindre, mer håndterbare deler.

- Mønstergjenkjenning - gjenkjenne og utnytte likheter
- Abstrahere/Abstraksjon - fjerne unødvendige detaljer
- Evaluering - vurdere resultat og prosessen



Figur 1 Figur 2 Figur 3

Figur 1: 1
Figur 1: $1 + 4 = 1^2 + 2^2$
Figur 3: $1 + 4 + 9 = 1^2 + 2^2$
⋮ Figur n: $1^2 + 2^2 + 3^2 + 4^2 + 5^2 + 6^2 + 7^2 = 140$.



Figur 1 Figur 2 Figur 3

c)

Figur 1: 1
Figur 1: $1 + 4 = 1^2 + 2^2$
Figur 3: $1 + 4 + 9 = 1^2 + 2^2$
⋮ Figur n: $1 + 4 + 9 + 16 = 1 + 2^2 + 3^2 + ... + n^2 \leq 1000$
Her kan vi prøve forskjellige n verdier:
$1 + 2^2 + 3^2 + ... + 10^2 = 385 \leq 1000$
$1 + 2^2 + 3^2 + ... + 11^2 = 506 \leq 1000$
$1 + 2^2 + 3^2 + ... + 12^2 = 650 \leq 1000$
$1 + 2^2 + 3^2 + ... + 13^2 = 819 \leq 1000$
$1 + 2^2 + 3^2 + ... + 14^2 = 819 \leq 1015$
Dermed figur nr. 13 første ledd har antall mindre enn 1000 klosser, og : $100 - 819 = 181$ klosser er igjen.

d) Programkode:

```
N = int(input("Hvor mange ledd vil du ta med i summen? "))


n = 1  # Definerer n og Sum ved start
Sum = 0
Nmaks=1000

# Vil regne ut summeringen til summen er mindre eller lik 20
for i in range(N):
    Sum += n**2
    n += 1

print('Summen til',N,'første ledd er', Sum)
print()
n = 1  # Definerer n og Sum ved start
Sum = 0
Nmaks=1000

# Vil regne ut summeringen til summen er mindre eller lik 20
while Sum <= Nmaks:
    Sum += n**2
    n += 1
    print('Summen til', n-1, 'første ledd er', Sum)

print()
```

```
print('HUSK: Summen passerer', Nmaks,' etter', n-1, 'ledd. Da er
    summen', Sum)
```

Modellering - testing

Ved å kjøre programmet får vi:

```
Hvor mange ledd vil du ta med i summen? 7
Summen til 7 første ledd er 140

Summen til 1 første ledd er 1
Summen til 2 første ledd er 5
Summen til 3 første ledd er 14
Summen til 4 første ledd er 30
Summen til 5 første ledd er 55
Summen til 6 første ledd er 91
Summen til 7 første ledd er 140
Summen til 8 første ledd er 204
Summen til 9 første ledd er 285
Summen til 10 første ledd er 385
Summen til 11 første ledd er 506
Summen til 12 første ledd er 650
Summen til 13 første ledd er 819
Summen til 14 første ledd er 1015

HUSK: Summen passerer 1000  etter 14 ledd. Da er summen 1015
```

Det finnes sikkert flere måter å svare på denne delen. Vi kan bruke denne for å sjekke svare for forrige oppgavedel: $100 - 819 = 181$ klosser er igjen.

Fasit 3.9.1:

a) 2+.5

b) 2.5

c) SyntaxError. Legg merke til at det mangler en apastrof etter 3.

d) Print skal være print.

e)
```
5+7
```

f) Print skal være print.

g)
```
5+7
```

print() legger en ekstra linje i resultatet.

h) Svaret blir 6.6. Legg merke til at 18/5 gir 3.6 og heltall divisjonen 18//5 gir 3) .

i)
```
5 10 15
```

j)
```
0 2 4 6
```

k)
```
5 4 3 2
```

l)

```
1, 2, 3
```

Fasit 3.9.2:

a) Programmet leser inn radiusen til en sirkel fra brukeren og regner og skriver ut arealet til en sirkel. Svaret blir 3.14

b)

```
import numpy as np

r= float(input('Skriv inn radiusen til en sylinder: '))

h= float(input('Skriv inn høyden til en sylinder: '))

def V(r,h):
    return np.pi*r**2*h

print(f' Volumet til en sylinder med radius {r} og {h} er lik {V(r,h
    )}')
```

Fasit 3.9.3:

En drosjeselskap har følgende regler for å regne reisekostnader:

- Hvis reiselengden er mindre enn 5 km., er fast pris 250 kr.
- Fast start pris er 200 kr. For hver km. regnes 10 kr.
- Hvis antall passasjerer er mer enn 4, må bestilles maxi taxi som koster 1.25 ganger mer.

Skriv en programkode som leser inn reiselengden og antall passasjerer fra brukeren, og regner ut reisekostnaden og skriver ut resultatet.

```
x = float(input("Hvor mange sms er sendt? "))

# kostnad regning
if x<500:
    kostnad=100
else:
    kostnad = 50 + x*.1
# Hvis antall sms >=1000, er kostnaden 150 kr.
    if x >= 1000:
        kostnad = 150
# Den totale kostnaden
print(f'Mobilkostnaden for {x} sms er Kr.{kostnad:.2f}')
```

Fasit 3.9.4:

a) La f være funksjonen gitt ved $f(x) = x^3 + 3x - 1$. Programkode som setter opp verditabell for x-verdiene mellom -2 og 2 (med å dele opp intervallet med steglengde $= 0.01$), og tegner grafen til f.

b)

```
import numpy as np
import matplotlib.pyplot as plt

xstart = float(input('Skriv inn nedre grense for x: '))
xslutt = float(input('Skriv inn øvre grense for x: '))
steglengde  = float(input('Skriv inn steglengden: '))
```

```
7
8  def f(x):
9      return x**3 + 3*x - 1
10
11 x = xstart
12 y = f(x)
13
14 while x <= xslutt:
15     x += steglengde
16     y = f(x)
17     print(f'{x:.3f} {y:.3f}')
18
19 x = np.arange(xstart, xslutt, steglengde)
20 y = f(x)
21
22 plt.plot(x,y, '--')
23 plt.grid()
24 plt.xlabel('x')
25 plt.ylabel('y')
26 plt.legend(['f(x)'])
27 plt.show()
```

c) Ja. Her deles intervallet i 401 punkt, det vil si i 400 deler.

Fasit 3.9.5:
Programmet bestemmer middelverdi og standardavvik (frihetsgrad n) for datasettet *data*.

Fasit 3.10.1:

a)
```
1 2  3  4  5  6  7  8  9  10
2 1  2  3  4  5  6  7  8  9
```

I programkode 1 kommer *print*-kommandoen like etter at x har blitt oppdatert med +1. Når 1 til 9 leses inn for x, det blir listet opp fra og med 2 til og med 10. I proramkode 2 *print*-kommandoen kommer like før at x har blitt oppdatert med +1. Når 1 til 9 leses inn for x, det blir listet opp fra og med 1 til og med 9.

b) Programkode 1, $m = 1$, $n = 10$. Programkode 2, $m = 1$, $n = 10$. da får vi samme svar som while-løkkene:

```
1 2  3  4  5  6  7  8  9  10
2 1  2  3  4  5  6  7  8  9
```

Fasit 3.10.2:

a) I situasjoner der vi ikke helt vet hvor lenge vi vi kjøre en løkke, while kan brukes. En for-løkke brukes når vi vet på forhånd hvor mange ganger løkken kan gå.

b)
```
1 Etter 45 måneder blir lånebeløpet 15189.51639445015
```

c)
```
1 vekstfaktor = 1.015
2 Antallmåneder = 0
3 y= 5000
4 sluttverdi=10000
5
```

```
while y < sluttverdi:
    Antallmåneder += 1
    y = y*vekstfaktor

print("Det tar", Antallmåneder, " måneder å spare opp", y)
```

Med å kjøre programmet får vi:

```
Det tar 47  måneder å spare opp 10066.395513857045
```

d) For 10. klasse elever kan dette forklares ved: $5000(1.025)^{45} \simeq 15189.5$
For videregåendeskole elever kan dette forklares ved: $5000(1.025)^{x} \simeq 15000$ som gir $1.025^{x} = 15000/5000 = 3$ og dermed $x = log3/log1.025 \simeq 44.49$ måneder

Fasit 3.10.3:

a) Vekstfaktor er 0.82 ($1 - 0.18 = 0.82$) og verdi = 350 000.

```
0 350\,000
1 287000.0
2 235340.0
3 192978.8
4 158242.61599999998
Etter 4 år er verdien mer enn halvert og prisen er
   158242.61599999998
```

b) For 10. klasse elever kan dette forklares ved: $350\,000(0.82)^{3} \simeq 192\ 979$, $350\,000(0.82)^{4} \simeq 158\ 243$ og $350\,000(0.82)^{3.5} \simeq 174\ 749,7$

For videregåendeskole elever kan dette forklares ved: $350\,000(0.82)^{x} = 175\,000$ som gir $082^{x} = 0.5$ og dermed $x = log0.5/log0.82 \simeq 3.49$ år.

Fasit 3.10.4:

a) Først angis startverdiene for låneverdi, antall år og rentefot (L = 2 500 000, t= 20, p= 0.025), Avdrag settes opp som låneverdi/antall år, og deretter regner ut terminbeløp i en løkke der regnes ut først renten, og for hver iterasjon, trekkes avdrag fra låneverdi for å bestemme den nye låneverdien. Til slutt skrives terminbeløpet ut. avdrag =L/t
for i in range(1,t+1): rente = L*p terminbeløp = avdrag + rente L = L - avdrag
print(i,int(terminbeløp))

b)

```
1 187500
2 182812
3 178242
4 173786
5 169441
6 165205
7 161075
8 157048
9 153122
10 149294
11 145561
12 141922
13 138374
14 134915
15 131542
```

```
16 16  128253
17 17  125047
18 18  121921
19 19  118873
20 20  115901
```

c)

```
p= 0.03
```

Fasit 3.10.5:

a) $\frac{T}{1{,}05} + \frac{T}{(1{,}05)^2} + \frac{T}{(1{,}05)^3} + \cdots + \frac{T}{(1{,}05)^{16}} = 100\,000.$

b) $T = \frac{100\,000 \cdot 0{,}05}{1 - (1{,}05)^{-16}} \simeq 7192$

c) $\frac{T}{a} + \frac{T}{a^2} + \frac{T}{a^3} + \cdots + \frac{T}{a^{n \cdot t}} = L.$

d) $T = \frac{p \cdot 0{,}01 \cdot L}{\left(1 - (1 + p/100)^{-n*t}\right)}$

e) Program:

```
# Termin.py beregner terminbeløpet gitt L (lånebeløp), P (rentefot)
    og antall termin.
L= float(input('Hva er låneverdien? '))
P = float(input('Hva er rentefot? '))
AntallTermin = float(input('Hvor mange termin skal ha betales pr. år?
    '))
Laanetid = float(input('Hva er nedbetalingstid?'))
def LaaneKalk(L, P, AntallTermin, Laanetid):
    Teller = (P / 100) * L
    Nevner = (1 + P / 100) ** (-1 * (Laanetid* AntallTermin))
    return str(round((Teller / (1 - Nevner)),2))

print('Terminbelopet er: ' + LaaneKalk(L, P, AntallTermin, Laanetid))
```

Ved å kjøre programmet, får vi:

```
Hva er låneverdien? 1000000
Hva er rentefot? 2
Hvor mange termin skal ha betales pr. år?4
Hva er nedbetalingstid?15

Terminbelopet er: 7191.99
```

Fasit 3.10.6:

Programkode:

```
farge= input("Hvilken farge ser du på trafikklyset (rød, grønn, oransje
    )?")
if farge == "rød":
print("STOP")
elif farge == "oransje":
```

```
print("Klar å kjøre")
else:
print("Kjør!")
```

Fasit 3.10.7:
Programkoden spør leseren å skrive et partall, og deretter sjekker om dette tallet er partall, og bestemmer summen til alle partall til og med dette tallet. Svaret blir 6 $(0 + 2 + 4 = 6)$.

```
tall = int(input("Skriv inn et oddetall: "))

if tall%2 != 0:
    print("Tallet er ikke et oddetall!")
else:
    sum = 0
    i = 1
    while(i <= tall):
        sum = sum + i
        i = i + 2

print(sum)
```

Svaret blir 9 $(1 + 3 + 5 = 9)$.

Fasit 3.10.8:
Programkoden:

```
km = float(input("Hva er reiselengden (KM): "))
p= int(input("Antall passasjerer: "))
# kostnad regning
if km<5:
    kostnad=250
else:
    kostnad = 200 + km*10
# Hvis antall passasjerer er mer enn 4, da må bestille maxi-taxi som
   koster 1.25 ganger mer.
    if p >= 5:
        kostnad = kostnad * 1.25
# Den totale kostnaden
print(f'Reisekostnaden for {p} passasjerer er Kr.{kostnad:.2f}')
```

Fasit 3.10.9:

a) Resultatet blir:

```
A
A
B
B
B
```

b) Resultatet blir:

```
A
B
B
```

c) Resultatet blir:

```
A
B
B
B
B
A
B
B
B
B
```

Fasit 3.10.10:

a)

```
for i in range(3):
    print('A')
for i in range(2):
print('B')
```

```

for i in range(3):
    print('A')
    for i in range(2):
        print('B')
```

b) **Fasit 3.10.11:**

a) Resultatet blir:

```
31
32
33

41
42
43
```

print() skriver ut en ekstra linje.

b) Resultatet blir:

```
313233
414243
```

print() skriver ut en ekstra linje.

Fasit 3.10.12:

a) Resultatet blir:

```
1 2
2 3
3 4
```

b) Resultatet blir:

```
1 2
3 4
5 6
```

c) Resultatet blir:

```
1
1 2
1 2 3
1 2 3 4
```

d) Resultatet blir:

```
101
102
103
201
202
203
```

print() skriver ut en ekstra linje.

Fasit 3.10.13:

a) Her skulle π hentes fra en pakke som numpy:

```
import numpy as np
def Vsylinder(r,h):
V=np.pi*r**2*h
return V

V=Vsylinder(3,5)

print(V)
```

b) Programkode:

```
import numpy as np
def Vkule(r):
V=4*np.pi*r**3/3
return V

V=Vkule(4)
print(V)
```

Fasit 3.10.14:

a) Koden beregner arealet til en trekant med lengden til grunnlinjen g og lengden til høyden h. Resultatet blir `15.0`.

b)

```
def Atrapes(a,b,h):
A=(a+b)*h/2
return A

A=Atrapes(3,5,4)
print(A)
```

Resultatet blir `16.0`.

Fasit 3.10.15:

```
def f(x):
    return -x**3+x+2

xstart = -2
xslutt = 3
xsteg = .25

xverdier = []
yverdier = []

x = xstart
y = f(x)

while x <= xslutt:
    xverdier.append(x)
    yverdier.append(y)
    x = x + xsteg
    y = f(x)
    print(xverdier, yverdier)
print(xverdier)
print(yverdier)
```

Fasit 3.10.16:

a)

```
from pylab import *

def kokepunkt(h):
    KT = 100 - 0.0032*h
    return KT

kokepunktE=kokepunkt(8848)

print("Vannskoketid på Everest er:",kokepunktE)
```

b) Programkode:

```
from pylab import *

def kokepunkt(h):
    KT = 100 - 0.0032*h
    return KT

kokepunktE=kokepunkt(8848)

print("Vannskoketid på Everest er:",kokepunktE)

def koketid(h,Tplomme,T0):
    Tvann= kokepunkt(h)
    A = 3.75
    tid = A*log( (2*(Tvann-T0)) / (Tvann-Tplomme) )
    return tid

```

```
def koketid(h,Tplomme,T0):
    Tvann= kokepunkt(h)
    A = 3.75
    if (Tvann-Tplomme)>0:
        tid = A*log( (2*(Tvann-T0))/(Tvann-Tplomme) )
        print("Det tar",tid,"minutter å koke et hardkokt egg på Everest")
        return tid
    else:
        print("Det er umulig å få den ønskede temperaturen på plommen")
```

Fasit 3.10.17:

Programmet plukker to tilfeldige tall mellom 1 og 99 og spør brukeren hva summen er. Hvis brukeren svarer et svar mindre enn summen, får man tilbakemelding: *Feil! Rett svar er mer enn dette svaret*. Hvis brukeren svarer et svar mer enn summen, får man tilbakemelding: *Feil! Rett svar er mindre enn dette svaret*. Hvis svaret er rett, får man tilbakemelding: Bra! Rett svar er svaret. Du klarte det etter et antall forsøk.

Fasit 3.10.18:

Programkoden.

```
f = open("fil1.txt", "a")
f.write("\n Vi ses snart!")
f.close()
#åpen og les fra filen
f = open("fil1.txt", "r")
print(f.read())
```

Hvis vi ikke tar med *f.close()*, filen er åpen hver gang vi kjærer og teksten dukker opp flere ganger når vi kjører programmet.
Vi kan slette alt innhold i filen med åpen filen for å skrive, og ta kommandoen: *f.truncate(0)*

```
f = open('fil1.txt', 'a')
# eller f = open('fil1.txt', 'w')

f.truncate(0)
```

Fasit 3.10.19:

a) Vi må ta med plot og pandas bibliotekene skriv i begynnelsen av filen.

```
import matplotlib.pyplot as plt
import pandas as pd
```

b) Generell kommentar: i programkoden leses data fra filen 'tekopp-temperatur.csv'. Hver linje i filen er en temperatur-måling i en te-kopp i løpet av en time (3600 sekunder) som er plassert i et rom med temperatur $T_o = 22$ Grad celsius. Her bruker vi Newtonsavkjølingslov for å studere temperaturendringen i vannkoppen. Kommentar linje for linje klarer du selv. Det er brukt Newtonsavkjølingslov $\frac{dT}{dt} = -k(T - T_o)$.

Fasit 3.10.20:

a) Programmet bestemmer middelverdi og emprisk standardavvik (frihetsgrad $n - 1$) for datasettet *data*.

b) Da kan data i programmet erstattes med denne listen og endre *data* til *liste* i programmet.

Fasit 3.10.21:
Programmet bestemmer middelverdi og emprisk standardavvik (frihetsgrad n) for datasettet *data*.

Fasit 3.10.22:

a) Resultatet blir:

```
Programmet simulerer et myntkast et antall ganger som brukeren har
    valgt og teller opp hvor mange ganger hver av utfallene (1, 2, $\
    codts$, 6) forekommer.
```

Programkoden:

```
import random

antallkast= int(input("Hvor mange ganger vil du kaste mynten? "))

antallkron = 0
antallmynt=0

for i in range (antallkast):
    kast = random.randint(0,1)
    if kast == 1:
        antallkron +=1
#       antallmynt=antallkast - antallkron
    else:
        antallmynt +=1

print(f"For {antallkast} tilfeldige kast ble {antallkron} kron og {
    antallmynt} mynt på ")
```

b) Programkoden:

```
import random

antallganger= int(input("Hvor mange ganger vil du spille stein-saks-
    papir?"))

antallstein = 0
antallsaks=0
antallpapir=0

for i in range (antallganger):
    kast = random.randint(0,2)
    if kast == 0:
        antallstein +=1

    elif kast == 1:
        antallsaks +=1
    else:
        antallpapir +=1

#       antallpapir=antallganger - (antallstein+antallsaks)
```

```
20 print(f"For {antallganger} tilfeldige stein-saks-papir ble {
       antallstein} stein, {antallsaks} saks og {antallpapir} papir.")
```

Dette heftet vil fremme læreglede for algoritmisk tenkning, modellering og problemløsning, og vil bruke programmering til å vise hvordan man kan løse oppgaver og problemer fra grunnleggende og brukerkurs i matematikk i universitetstrinnet. Vi håper at dette heftet bidrar til at foreleseren og læreren finner sin egen programmeringsdidaktikk for å tilrettelegge programmeringsundervisningen på en engasjerende, morsom og relevant måte.

Struktur og progresjon i oppgavene skal forberede leseren til å kunne løse utfordrende problemstillinger og tilegne seg utholdenhet, kreativitet, selvtillit, og vil bidra til matematisk tenkning.

Problem: Regn ut $\sqrt{1{,}06}$ uten å bruke kalkulator.

Matematisk modell: tangentmetoden:
$f(x) = \sqrt{x+1}$, $f(0) = 1$, $\quad f'(x) = \dfrac{1}{2\sqrt{x+1}}$, $f'(0) = \dfrac{1}{2}$.
$\sqrt{x+1} \simeq 1 + \dfrac{1}{2}x$ $\quad$ når x er nær 0 ($x \simeq 0$).
$\sqrt{1{,}06} \simeq 1 + \dfrac{0{,}06}{2} = 1{,}03$.

Bibliografi

[1] Amir M. Hashemi and Arne Hole (2025). Matematikk for ingeniørstudenter, 1. utgave. Fagnbokforlaget 2025, ISBN 9788245052114.

[2] Archer, Louise; DeWitt, Jen; Willis, Beatrice (2013). "Adolescent boys' science aspirations: Masculinity, capital, and power". Journal of Research in Science Teaching. 51 (1): 1–30.

[3] Alseth, B., Breiteig, T. Brekke, G. (2003). Endringer og utvikling ved R97 som bakgrunn for videre planlegging og justering - matematikkfaget som kasus. Hentet fra http://hdl.handle.net/11250/2439972(Kap. 2, s. 11-43)

[4] Hårberg, Guri Bente. Grønlid, Gro Nedberg. CC CC-BY-SA-4.0 F. Hentet fra https://ndla.no/subject:1:2cbe8089-7d7b-407f-8f04-fbfdc116abc1/topic:3:183771/topic:2:184796/resource:1:16864

[5] Katz, I., Kaplan, A., Gueta, G. (2009): Students' Needs, Teachers' Support, and Motivation for Doing Homework: A Cross-Sectional Study, The Journal of Experimental Education, 78:2, 246-267

[6] Katz, I., Kaplan, A., Gueta, G. (2010). Students' needs, teachers' support, and motivation for doing homework: A cross-sectional study. Journal of Experimental Education, 78(2), 246–267. Hentet fra https://doi.org/10.1080/00220970903292868

[7] Jakobsen, Renate E. (2019) Programmering i matematikk –muligheter og utfordringer (Masteroppgave, UiA). Henter fra https://uia.brage.unit.no/uia-xmlui/bitstream/handle/11250/2646381/Renate 20Elise 20Jakobsen.pdf

[8] Kunnskapsdepartementet. (2011). Fra Matteskrekk tilmattemestring.Oslo, Hentet 15/4.2021 fra http://www.regjeringen.no/nb/dokumentarkiv/stoltenberg-ii/kd/Nyheter-og-pressemeldinger/nyheter/2011/fra-matteskrekk-til-mattemestring1.html?id=652802

[9] Kunnskapsdepartementet (2013). Læreplan i matematikk fellesfag. Hentet fra https://www.udir.no/kl06/mat1-04

[10] Kunnskapsdepartementet (2018). Kjerneelementer i fag. Hentet fra https://www.regjeringen.no/no/aktuelt/fornyer-innholdet-i-skolen/id2606028/

[11] Læreplan i matematikk fellesfag (MAT1-04)

[12] Matematikk 1–10 (MAT01-05). Hentet fra https://www.udir.no/lk20/mat01-05/kompetansemaal-og-vurdering/kv14?lang=nob

[13] Oversikt over deltakere på kodetimen: http://kidsakoder.no/kodetimen/

[14] Papert, Seymour. Mindstorms: Children, computers, and powerful ideas. Basic Books, Inc., 1980. Hentet fra http://worrydream.com/refs/Papert 20- 20Mindstorms 201st 20ed.pdf

[15] Rapport og planer (2019-2020), Nasjonalt Senter for Realfagsrekuttering. Hentet fra https://realfagsrekruttering.no/om-oss/rapport-og-planer

[16] Rapport og planer, Nasjonalt Senter for Realfagsrekuttering. Hentet fra https://realfagsrekruttering.no/om-oss/rapport-og-planer

[17] Rapport og planer (2015-2016), Nasjonalt Senter for Realfagsrekuttering. Hentet fra https://realfagsrekruttering.no/om-oss/rapport-og-planer

[18] Ryan, R. M., Deci, E. L. (2000). Self-determination theory and the facilitation of intrinsic motivation, social development, and well-being. American Psychologist, 55(1), 68–78. Hentet fra https://doi.org/10.1037/0003-066X.55.1.68

[19] Sevik, Kristine m.fl. (2016). Programmering i skolen. Notat fra Senter for IKT i utdanningen. Hentet fra https://www.udir.no/globalassets/filer/programmering-i-skolen.pdf

[20] Skinner, E., Furrer, C., Marchand, G., Kindermann, T. (2008). Engagement and disaffection in the classroom: Part of a larger motivational dynamic? Journal of Educational Psychology, 100(4), 765–781. Hentet fra $https://doi.org/10.1037/a0012840$

[21] Syslak, Irmelin. Relasjonens betydning for motivasjon og læring, ENT3R-mentor mentorkurs, presentasjon, 2023.

[22] Udir, Notat nr. 2 - Programmering i skolen. November 2016104. Programmering som kompetanse for det 21. århundre. https://www.udir.no/kvalitet-og-kompetanse/profesjonsfaglig-digital-kompetanse/notat-om-programmering-i-skolen/

[23] Utdanningsdirektoratet (2013). Læreplan i matematikk fellesfag. Hentet fra https://www.udir.no/kl06/MAT1-04

[24] Utdanningsdirektoratet (2017). Kjerneelementer i matematikk, men hvorfor programmering? Hentet fra https://udirbloggen.no/kjerneelementer-i-matematikk-men-hvorfor-programmering/

[25] Utdanningsdirektoratet (2019). Læreplan i matematikk 1.-10. trinn. Hentet fra https://www.udir.no/lk20/mat01-05

[26] Utdanningsdirektoratet (2019). Læreplan i matematikk fellesfag Vg1 teoretisk. Hentet fra https://www.udir.no/lk20/mat09-01?lang=nob

[27] Wentzel, K. R., Battle, A., Russell, S. L., Looney, L. B. (2010). Social supports from teachers and peers as predictors of academic and social motivation. Contemporary Educational Psychology, 35(3), 193–202. Hentet fra https://doi.org/10.1016/j.cedpsych.2010.03.002

[28] Wing, Jeannette (april 2008). «Computational thinking and thinking about computing». 2008 IEEE International Symposium on Parallel and Distributed.

[29] Wæge, Kjersti, Nostrai, Mona. Sentrale kjennetegn på god læring og undervisning i matematikk , Matematikksenteret.no